

Diario di bordo

Progetto d'esame — riproduzione di *Audio Sparse-Transformer* (Kavaki & Mandel, ICASSP 2025)

B031278 Deep Learning, Fall 2025 — MSc in Artificial Intelligence, Università di Firenze

Ultimo aggiornamento: 23 agosto 2026

A COSA SERVE QUESTO DOCUMENTO

Registro cronologico di **tutto** ciò che viene fatto sul progetto: decisioni e loro motivazione, verifiche eseguite, **errori commessi** e cambi di direzione. Serve per ripassare, per ricostruire *perché* una scelta è stata fatta mesi dopo, e per rispondere all'orale a domande del tipo «perché hai fatto così e non cosà».

Il piano di lavoro sta invece in `Pipeline_progetto_DL.pdf` : quello dice *cosa* fare, questo dice *cosa è stato fatto*.

LEGENDA

DECISIONE scelta presa, con motivazione · **VERIFICA** qualcosa controllato o misurato · **ERRORE** sbaglio commesso · **CORREZIONE** rettifica di qualcosa detto o fatto prima · **ARTEFATTO** file prodotto

COME SI AGGIORNA

Il sorgente è `docs/diario_di_bordo.html` . Dopo averlo modificato:

```
powershell -File scripts\build_docs.ps1
```

rigenera `Diario_di_bordo.pdf` e `Pipeline_progetto_DL.pdf` nella cartella di progetto, usando Edge in modalità headless (nessuna dipendenza da installare).

Giorno 1 — mercoledì 12 agosto 2026

Obiettivo della sessione: capire cosa chiede l'esame, cosa dicono i due paper, e definire un metodo di lavoro.

VERIFICA Analisi della pagina del corso

Letta integralmente la pagina ufficiale (ai.dinfo.unifi.it/teaching/dl_2025.html), salvata in locale. Estratti i vincoli d'esame che condizionano ogni scelta tecnica:

- progetto da **concordare a ricevimento** (lunedì 10:45–12:45, S. Marta): il docente dà i dettagli operativi;
- **nessuna relazione scritta** — si consegna solo codice più istruzioni di riproduzione, quindi il `README.md` è l'unico documento valutato;
- consegna su **repository privato Codeberg** con l'utente `dl.unifi` invitato; **mai i dati**, solo un link;
- presentazione di **30 minuti** con letteratura, derivazione tecnica ed esperimenti; domande individuali su tutto il programma;
- riduzione di dataset e modelli **esplicitamente autorizzata** se le risorse non bastano;
- esiste un **form Google per richiedere risorse di calcolo**.

Annotato l'avvertimento del docente sul leggere «other ancestors in the citation graph»: si rivelerà centrale il giorno 2.

VERIFICA Ricerca sui due paper assegnati

Gazneli et al. 2022 (EAT): arXiv ad accesso libero, codice ufficiale su github.com/Alibaba-MIIL/AudioClassification. Estratti architettura (conv 1D depthwise su waveform grezza + transformer), varianti EAT-S 5,3 M / EAT-M 25,5 M, le augmentation proposte (Mixup, FreqMix, PhaseMix, TimeMix/CutMix) e la ricetta di training (AdamW, one-cycle, EMA 0,995, label smoothing, BCE).

Kavaki & Mandel 2025: risultato **closed access**, nessun preprint arXiv, nessun codice pubblico. Semantic Scholar riporta `openAccessPdf: CLOSED` e reference list elisa dall'editore. Disponibile solo l'abstract: 96,88 % su Speech Commands V2 con -85,25 % di FLOPs rispetto a EAT-S.

ERRORE Ipotesi sbagliata sull'origine del metodo

Non potendo leggere il paper, è stata formulata l'ipotesi che le «regioni significative» dello spettrogramma derivassero dalla linea di ricerca precedente di Mandel sulle *importance map* — *Identifying Important Time-Frequency Locations* (Interspeech 2020) e *ImportantAug* (ICASSP 2022) — e che il decoding fosse un embedding di punti con encoding posizionale continuo, cioè lo spettrogramma trattato come point cloud.

L'ipotesi era sbagliata ed è stata smentita il giorno 2 leggendo il PDF. Il vero ancestor è SparseFormer. Vedi la voce del 17/08. L'ipotesi era stata comunque marcata come «da confermare, non da dare per buona»: la lezione è che un'ipotesi plausibile su un metodo non letto resta una congettura, e va tenuta separata dai fatti.

DECISIONE Struttura del lavoro in sei fasi

Definito un piano che mette **l'infrastruttura di misura prima del modello**. Motivazione: il claim da riprodurre è un *rapporto* accuratezza/FLOPs, non un'accuratezza; un contatore di FLOPs scritto dopo il modello tende a essere costruito per confermare il risultato atteso.

Definiti anche i **criteri di successo in anticipo**, per non poterli riscrivere a posteriori in base a quello che esce.

VERIFICA Ricognizione della macchina

GPU	NVIDIA GeForce RTX 5050 Laptop, 8151 MiB, driver 592.82 — Blackwell, sm_120
Python di sistema	3.12.4
Disponibili	miniconda, git — uv assente, PyTorch non installato

Verificati i due ambienti DL esistenti: `C:\ai\pytorch_env` (torch 2.9.1+cu128, Python 3.12.4) e conda `DLA2026` (torch 2.10.0, CUDA 13, Python 3.14.3). **Entrambi già compatibili con sm_120**. In entrambi mancavano però `soundfile` e `numba`, e `torchaudio` era nella variante `+cpu`.

DECISIONE Ambiente dedicato invece di riusare quelli esistenti

Scelto di creare un venv nuovo su Python 3.12 anziché usare `pytorch_env` o `DLA2026`. Tre motivi:

- **riproducibilità**: un `pip freeze` da un ambiente generalista con JAX, OpenCV, mediapipe e pyautogui produce un `requirements.txt` inutilizzabile, e l'ambiente è parte del deliverable;
- `DLA2026` appartiene a **un altro corso**: modificarlo rischia di romperne il lavoro;
- Python 3.14 di `DLA2026` taglierebbe fuori `numba`/librosa se dovessero servire.

ERRORE Generazione del PDF fallita al primo tentativo

Il primo tentativo di conversione HTML→PDF con Edge headless non ha prodotto alcun file. Causa: un flag inesistente (`--print-to-pdf-no-header`) e il profilo utente di Edge bloccato da un'istanza attiva. **Risolto** passando un `--user-data-dir` dedicato e terminando i processi `msedge` residui. La ricetta funzionante è ora in `scripts/build_docs.ps1`.

ARTEFATTO Prodotti i primi due file

- `Pipeline_progetto_DL.pdf` **v1.0** — 14 pagine: vincoli d'esame, analisi dei paper, pipeline in 6 fasi, protocollo sperimentale, piano di studio a 10 settimane, programma del corso come checklist, derivazioni da saper fare, struttura del repo;
- `requirements.txt` — installazione in due passi, con la motivazione di ogni pacchetto non ovvio.

Giorno 2 — lunedì 17 agosto 2026

Obiettivo: leggere il paper vero, correggere il piano, costruire l'infrastruttura della Fase 2.

VERIFICA I PDF dei paper sono arrivati

Trovati nella cartella: `2204.11479v5.pdf` (EAT), `Audio_Sparse-Transformer_for_Speech_Classification.pdf` (copia UNIFI da IEEE Xplore) e una cartella `paper pre progetto/` con ViT, *Attention is all you need* e **arXiv 2304.03768**.

CORREZIONE Ostacolo tecnico: lettura dei PDF

Lo strumento di lettura PDF richiedeva `poppler`, non installato e non installabile sul momento. **Aggirato** usando **PyMuPDF**, già presente nell'ambiente `conda DLA2026`, per estrarre il testo in un file temporaneo. Utile ricordarlo: gli ambienti esistenti sono una risorsa anche quando non li si usa per il progetto.

CORREZIONE L'ancestor era un altro — ipotesi del giorno 1 smentita

Letto il paper per intero. Il metodo **non** deriva dalla linea importance-map di Mandel. L'origine è **SparseFormer** (Gao et al. 2023, **arXiv:2304.03768**, *Sparse visual recognition via limited latent tokens*), un modello per ImageNet di cui questo lavoro è la trasposizione all'audio. Gli altri due riferimenti strutturali sono **Faster R-CNN** (parametrizzazione delle regioni) e **AdaMixer** (pesi di decoding adattivi).

Differenza sostanziale rispetto all'ipotesi: **le regioni non sono selezionate per saliency o energia — sono parametri appresi**, aggiustati iterativamente dai token stessi. E il transformer **non usa positional encoding**, perché i token non hanno ordine.

Nota: `2304.03768` era già nella cartella `paper pre progetto/`. L'ancestor giusto era a disposizione fin dall'inizio.

DECISIONE Cambio di direzione: il baseline non è EAT-S

Dalla Tabella 2 del paper emerge che il confronto *controllato* non è contro EAT-S ma contro il **dense-transformer** degli autori: stessa architettura che processa i frame temporali invece dei token latenti. Il numero di EAT-S è **citato dalla letteratura**, non rimisurato dagli autori.

Modello	Params	FLOPs	Accuracy
Sparse-transformer (N=4, P=36)	2,87 M	0,055 G	96,88 %
Dense-transformer	4,80 M	0,645 G	96,67 %
EAT-S (<i>citato</i>)	5,30 M	0,373 G	98,15 %

Conseguenza sul piano: la Fase 3 diventa «early convolution + dense transformer», che condivide tutto il codice a valle col modello sparso; la reimplementazione completa di EAT-S scende a **Fase 4-bis, opzionale**. Semplificazione netta e riduzione del rischio.

VERIFICA Il metodo, ricavato dal paper

N token latenti e N regioni $b=(x,y,w,h)$, **entrambi parametri appresi**; centri inizializzati su griglia, larghezze a metà della dimensione della feature map. Per $L_{rep}=3$ volte: aggiustamento della regione via `Linear(t)` con parametrizzazione esponenziale → campionamento di P punti con offset appresi e interpolazione bilineare → decoding adattivo con M_c e M_s generati da t , GELU, update residuo del token. Poi 8 transformer-encoder, average pooling, classificatore lineare.

Configurazione: $N=4$, $P=36$, $d_{\text{token}}=64$, $d_{\text{encoder}}=128$, $L_{\text{enc}}=8$. Training: AdamW lr $3 \cdot 10^{-4}$, $\beta=(0,9 \cdot 0,99)$, one-cycle, wd 10^{-5} , EMA 0,995, batch 64, augmentation *mixing* e *phasemix*.

Dettaglio critico: non si campiona dallo spettrogramma diretto — il gradiente attraverso l'interpolazione bilineare è troppo rumoroso con pochi punti. Si applica prima una *early convolution* e si campiona sul suo output.

VERIFICA Ambiguità trovate nel paper

Sei punti su cui il paper tace o si contraddice, tutti da dichiarare come assunzioni:

- **parametri dello spettrogramma mai dichiarati** (mel o lineare? n_{fft} ? hop? bin?) — il parametro libero più impattante;
- **contraddizione interna in §3.2:** feature «96 dimensional» ma «196 kernels» 7×7 stride 2;
- numero di layer della early convolution non dichiarato;
- numero di epoche non dichiarato;
- come il dense-transformer costruisce i frame dall'output della early conv;
- convenzione FLOPs vs MAC non dichiarata.

Osservazione critica aggiuntiva: nella Tabella 3 l'ablation sui token **non è monotona** (16 token $\rightarrow$ 97,53 %, 25 token $\rightarrow$ 97,20 %, 36 $\rightarrow$ 97,94 %) e i risultati sono su **singolo seed**. Suggerisce un rumore da seed di 0,3–0,4 punti, paragonabile a diverse delle differenze discusse nel paper. È la giustificazione sperimentale dei ≥ 3 seed.

CORREZIONE Stima sbagliata della dimensione della cache

Nella v1.0 del PDF la cache del training set era stimata « $\approx 1,7$ GB». Valore corretto: **2,72 GB** ($84\,843 \times 16\,000$ campioni $\times$ 2 byte). Con validation e test si arriva a $\sim 3,4$ GB. Corretto nel PDF e nel codice.

ARTEFATTO Pipeline_progetto_DL.pdf v2.0 — 16 pagine

§3 riscritta col metodo reale e le formule; §3.4 nuova con le ambiguità; Fase 3 riscritta e Fase 4-bis aggiunta; §5.1 con gli split esatti e i target di riproduzione; §8 con tre derivazioni in più; §12 con la bibliografia corretta.

ARTEFATTO Codice della Fase 2

```
src/data/speech_commands.py  download, split ufficiali, cache mmap
src/data/dataset.py          Dataset e DataLoader
src/data/features.py          log-mel su GPU
src/data/augment.py           mixing e phasemix su GPU
src/flops.py                  doppio contatore + latenza
src/utls.py                   seed, config, EMA
scripts/check_env.py          verifica ambiente
scripts/prepare_data.py       Fase 2 completa
configs/base.yaml             assunzioni marcate [ASSUNZIONE]
README.md                     .gitignore
```

Scelta di progettazione: `build_splits()` **fallisce di proposito** se i conteggi non danno esattamente $84\,843 / 9\,981 / 11\,005$. Sono i numeri dichiarati dal paper e coincidono con gli elenchi ufficiali speaker-disjoint: è un test di correttezza gratuito.

ERRORE Cancellazione accidentale di src/ e scripts/

Alle **16:54**, contestualmente alla creazione del venv `DL_F`, le cartelle `src/` e `scripts/` sono state eliminate insieme a un primo `.venv` — presumibilmente una multi-selezione sfuggita durante la pulizia del primo tentativo di ambiente. Metà del codice del progetto è sparita senza che ci fosse ancora un repository git.

Recuperate dal cestino e riverificate: tutti i file alle dimensioni originali, tutti ricompilano.

Lezione: il repository git va inizializzato *subito*, non «quando il codice sarà pronto». Un `git init` più un commit avrebbero reso l'incidente un non-evento.

CORREZIONE `.gitignore` non copriva il `venv`

Il `.gitignore` ignorava `.venv/`, ma l'ambiente si chiama `DL_F`: al primo `git add` sarebbero finiti sotto controllo di versione diverse migliaia di file. Aggiunta la riga `DL_F/`.

VERIFICA Installazione dell'ambiente

Verificata la raggiungibilità di **entrambi** gli indici pacchetti — `pypi.org` e `download.pytorch.org/whl/cu128` — prima di iniziare: se il secondo non risponde, pip ripiega silenziosamente sulla build CPU-only. Aggiornati pip (`24.0` → `26.2.1`), `setuptools` e `wheel`.

Installazione di torch avviata in background e **interrotta su richiesta**, ma il pacchetto era già stato installato per intero e funzionante. L'installazione è stata poi completata manualmente.

Esito di `scripts/check_env.py`:

```
torch 2.9.1+cu128 · CUDA 12.8 · capability (12, 0) · kernel di prova OK
arch compile: sm_70 ... sm_100, sm_120
torchaudio 2.9.1+cu128 · soundfile 0.14.0 · numpy 2.5.2
yaml 6.0.3 · fvc core 0.1.5 · einops 0.8.2 · optuna 4.9.0
=> ambiente pronto
```

VERIFICA Suite di test: 18 su 18 passati

Scritta `tests/test_pipeline.py`, eseguibile **senza dataset**: forme del front-end, invarianti delle augmentation, contatore di FLOPs, EMA, riproducibilità dei seed. Eseguita prima del download di 2,3 GB, per separare i bug dell'infrastruttura da quelli del modello.

Risultati notevoli:

- front-end conferma la forma `[B, 1, 64, 101]` — quel **101** è la T del modello denso, che il modello sparso riduce a **N=4** token: il fattore ~ 25 da cui viene il guadagno;
- **rapporto FLOPs/MAC verificato empiricamente**: su un `Linear(100→200)` senza bias, `fvc` riporta 20 000 e il contatore nativo di torch 40 000 — **esattamente 2,000000**. L'avvertenza di `src/flops.py` non è più un'ipotesi;
- EMA con decay 0,9 si sposta di esattamente $1 - \text{decay} = 0,1$: la media mobile insegue, non copia.

DECISIONE Sorgenti della documentazione dentro al progetto

I sorgenti HTML dei PDF stavano in una cartella temporanea legata alla singola sessione di lavoro, quindi non ritrovabili in futuro. Spostati in `docs/` e aggiunto `scripts/build_docs.ps1` per rigenerare entrambi i PDF con un comando. Ora i documenti sono manutenibili e versionabili come il resto del codice.

ERRORE `.gitignore`: `data/` nascondeva `src/data/`

Al primo `git add --dry-run` risultavano **18 file** invece dei 23 attesi: mancava tutto `src/data/`. Causa: in `gitignore` un pattern **senza barra iniziale matcha a qualsiasi profondità dell'albero**, quindi `data/` escludeva anche `src/data/` — cioè la pipeline dati completa, in silenzio.

Corretto in `/data/`, ancorato alla radice. Senza il controllo preventivo si sarebbe ottenuto un repository che *sembra* completo e che al primo clone su un'altra macchina non funziona. Stesso codice perso due volte nella stessa giornata, per cause diverse.

DECISIONE Repository git locale, e solo il necessario dentro

Inizializzato il repository (**locale, nessun remote, nessun invito**). Esclusi dal versionamento la pagina del corso salvata e i suoi asset — il solo `semantic.css` era il 90 % delle righe versionate — e la cartella `paper pre progetto/`, che contiene PDF altrui non ridistribuibili. Restano sul disco, fuori dal repo. Da 44 519 a poco più di 4 000 righe versionate.

Rimosso anche il trailer di co-autoria dai messaggi di commit: la cronologia riporta solo l'autore.

VERIFICA Fase 2 completata: dataset scaricato e cache costruita

Scaricati 2,43 GB, MD5 verificato, estratto. Gli split ufficiali tornano **esatti**:

```
[split] train      84843 (atteso 84843) OK
[split] validation  9981 (atteso  9981) OK
[split] test       11005 (atteso 11005) OK
```

Cache memmap costruita per tutti e tre gli split. Distribuzione delle classi nel training set: da 1 256 a 3 250 campioni per classe, cioè uno sbilanciamento di $\sim 2,6\times$ — non estremo, coerente con l'uso dell'accuratezza semplice come metrica.

ERRORE La memmap non si può picklare: `num_workers>0` andava in crash

Primo smoke test su dati veri: con `num_workers=0` tutto bene, con `num_workers>0` crash immediato con `OSError: [Errno 22] Invalid argument` e `UnpicklingError: pickle data was truncated`.

Causa: su Windows il DataLoader usa il metodo `spawn`, che **pickla l'oggetto Dataset** per inviarlo a ogni worker. La memmap era un attributo dell'istanza, quindi pickle tentava di serializzare l'intero array da 2,7 GB.

Correzione: apertura pigra della memmap tramite property e `__getstate__` che la rimuove dallo stato serializzato. Ogni worker riapre la propria vista sullo stesso file — che è il comportamento voluto, perché il sistema operativo condivide le pagine fra processi. Aggiunto un **test di regressione** che pickla il Dataset e verifica che il blob resti sotto i 100 KB.

Nota: il default di `configs/base.yaml` era `num_workers: 4`, quindi il bug sarebbe esploso al primo training.

ERRORE Smoke test scritto male: le augmentation sembravano rotte

Il primo smoke test prendeva i primi 64 campioni della cache e riportava «classi attive per campione: 1,00», come se mixing e phasemix non mescolassero nulla.

Non era un bug delle augmentation ma del test: la cache è scritta **in ordine di classe** (34 cambi di etichetta su 84 843 campioni), quindi i primi 64 elementi sono tutti *backward* e mescolarli fra loro produce un target a una sola classe. Rifatto il test con indici casuali: **1,83 classi attive in media, massimo 2** — esattamente il comportamento atteso.

Conseguenza operativa da ricordare: la cache è ordinata per classe, quindi qualsiasi iterazione *senza shuffle* produce batch a classe singola. Va bene per la valutazione, sarebbe disastroso per il training.

VERIFICA Smoke test end-to-end e prestazioni

Misura	Valore	Commento
Front-end	[64, 16000] → [64, 1, 64, 101]	media 0,000 · std 0,9999 · tutti finiti
Throughput dati	~24 600 campioni/s	~3,5 s per epoca di soli dati
Picco memoria GPU	59 MB	su 8 GB: margine enorme
Clip zero-paddate	11,6 %	registrazioni più corte di 1 s

Conclusione: **la pipeline dati non sarà mai il collo di bottiglia**. Il costo di un'epoca sarà interamente il modello. Suite di test a **19/19**.

VERIFICA Fase 3, moduli 1–3: front-end, encoder, modello denso

Scritti `src/models/frontend.py` (early convolution), `transformer.py` (encoder reimplementato con einops) e `dense.py` (baseline). Tutti parametrici sulle ambiguità del paper, così le scelte si fanno per misura e non per intuizione.

L'encoder è scritto per esteso invece di usare `nn.MultiheadAttention`: la consegna chiede una reimplementazione autonoma e all'orale va spiegato ogni passaggio. **Verificato copiando i nostri pesi dentro l'implementazione di PyTorch: differenza massima delle uscite esattamente 0.0** — non plausibilità, equivalenza.

VERIFICA Invarianza a permutazioni dimostrata sperimentalmente

senza positional encoding: permutando il tempo, l'uscita cambia di $2.4e-07$
 con positional encoding: permutando il tempo, l'uscita cambia di 0.083

Il $2,4 \cdot 10^{-7}$ è rumore di virgola mobile: **senza codifica posizionale il transformer è esattamente invariante all'ordine**. È la giustificazione formale dell'esperimento «con e senza pos. encoding» sul modello denso, e la risposta da dare all'orale alla domanda «perché serve il positional encoding»: non «aiuta», è che senza il modello non può in linea di principio distinguere l'ordine.

VERIFICA Costo di training misurato: la profondità costa più della larghezza

configurazione	param	s/epoca	100 epoche
d=128 L=8	1,79 M	15,6	26,0 min
d=224 L=8	5,19 M	23,7	39,5 min
d=256 L=8	6,72 M	26,3	43,8 min
d=128 L=24	4,96 M	43,8	73,0 min

Le ultime due righe hanno praticamente gli stessi parametri, ma quella profonda impiega **1,85× il tempo**. A queste dimensioni la GPU è limitata dal *lancio dei kernel*, non dal calcolo: 24 layer significano tre volte le chiamate CUDA, ciascuna su matrici troppo piccole per saturare gli SM.

È la dimostrazione concreta, sul nostro hardware, che **i FLOPs non predicano la latenza** — osservazione da portare in presentazione. Conseguenza operativa: nella ricerca della configurazione si preferisce la larghezza alla profondità.

DECISIONE Ricostruire la configurazione del denso usando i numeri come vincoli

Il paper non dichiara **alcun** iperparametro del dense-transformer: solo 4,80 M parametri e 0,645 G FLOPs. Assemblare secondo la Tabella 1 (che descrive lo *sparso*) dà ~1,8 M, cioè un fattore 2,7 di scarto.

Scelto di trattare i due numeri come **vincoli** e cercare le configurazioni che li soddisfano entrambe. Costo aggiuntivo misurato: +13 minuti per run da 100 epoche. In cambio i nostri FLOPs diventano confrontabili con quelli del paper, che è l'intero punto della riproduzione.

VERIFICA Ricerca su 270 configurazioni — due deduzioni

Enumerate lettura dei canali × stride del pooling × formazione della sequenza × d × profondità, contando parametri e costo con entrambi i contatori.

Deduzione 1 — il pooling agisce solo sull'asse frequenza. Tutte le configurazioni vicine al costo dichiarato hanno `pool_stride=(2,1)`, cioè **N = 51 frame**. Con il pooling su entrambi gli assi (N = 26) il costo si ferma a ~0,30 G contro gli 0,645 richiesti: il fattore 2 non è recuperabile con altri gradi di libertà.

Deduzione 2 — il paper riporta FLOPs, non MAC. Tutte le configurazioni che si avvicinano a 0,645 lo fanno nella colonna FLOP (0,53–0,66), mentre i corrispondenti MAC stanno a 0,26–0,33, cioè metà. **È la calibrazione della convenzione che serviva**, ottenuta senza addestrare nulla.

Miglior candidato congiunto:

```
c196_proj96 · pool (2,1) · pool_freq · d=256 · L=6 · N=51
4.801.807 parametri    (dichiarati 4,80 M -> scarto +0,0%)
0,562 GFLOP           (dichiarati 0,645 -> scarto -12,9%)
```

I parametri combaciano quasi esattamente; resta uno scarto del 13 % sul costo. **Ipotesi:** il paper dice «convolutional layers» al plurale, mentre il nostro stem ne ha uno solo; un secondo strato convolutivo aggiungerebbe circa l'ordine di grandezza mancante. Da verificare — ma senza inseguire il numero: aggiustare l'architettura finché il valore combacia significherebbe fare overfitting su una cifra riportata senza convenzione dichiarata.

Giorno 3 — martedì 18 agosto 2026

Obiettivo: chiudere con evidenze i punti deboli emersi dalla revisione critica, prima di scrivere il training loop.

ERRORE Nella prima ricerca avevo escluso lo stem dalla griglia

Le 270 configurazioni del 17/08 avevano tutte lo stem a una sola convoluzione: `stem` non era fra i gradi di libertà. La deduzione «il pooling agisce solo sulla frequenza» poteva quindi essere un artefatto di quella omissione.

Rifatta la ricerca su 432 configurazioni includendo lo stem alla Xiao. La deduzione regge: la famiglia $N=26$ si ferma a 0,451 GFLOP contro i 0,645 richiesti, e nemmeno lo stem più pesante recupera il fattore 2. Un test che avrebbe potuto smentire il risultato lo ha invece rafforzato.

ERRORE Avevo ottimizzato un parametro che il paper aveva già dato

La ricerca trattava la profondità L come incognita e sceglieva $L=6$, che fitta i parametri a +0,0 %. Ma il paper dichiara **8 transformer-encoder** e descrive il denso come «the same architecture but processes a sequence of time frames»: la profondità era vincolata.

È l'errore classico del fitting: ottimizzare su ciò che è già noto. Imponendo $L=8$ si perdono 2 punti percentuali sui parametri e si guadagna coerenza col testo, che all'esame vale molto di più.

VERIFICA E1 — quanto varia il livello di registrazione

percentili RMS su 20.000 clip: p1 -40,6 dB p50 -24,4 dB p99 -12,0 dB
escursione p1-p99: 28,5 dB -> fattore 27x in ampiezza

La premessa della normalizzazione per-campione è vera e non di poco: la stessa parola arriva al modello con ampiezze che differiscono di oltre un ordine di grandezza.

VERIFICA E2 — quanti bin mel? Il parametro «più impattante» è quasi piatto

Sonda lineare (regressione logistica su log-mel mediati su 8 finestre, 12 000 train / 4 000 val):

32 bin	41,60%	40 bin	41,93%	64 bin	42,18%
80 bin	42,40%	128 bin	42,03%		

0,8 punti di escursione su un fattore 4 di risoluzione, e curva non monotona (128 fa peggio di 80). Il grado di libertà che avevamo dichiarato come il più impattante dell'intera riproduzione è in realtà quasi irrilevante — almeno per l'informazione linearmente decodificabile.

Confermati i **64 bin**: a 0,22 punti dal migliore, differenza dentro il rumore, e unico valore che dopo lo stem dà $F=16$, potenza di due, utile alla geometria del campionamento sparso.

ERRORE E3 — la normalizzazione che avevo aggiunto non è supportata dai dati

sonda lineare CON normalizzazione : 42,33%
sonda lineare SENZA normalizzazione : 42,83%

Avevo introdotto la normalizzazione per-campione dello spettrogramma convinto che aiutasse, motivandola con la grande escursione di livello. L'escursione c'è (E1), ma la normalizzazione **peggiora** la sonda di mezzo punto: il livello assoluto porta evidentemente un po' di segnale utile.

Declassata da default silenzioso a flag di configurazione e ablation dichiarata. Nessuna delle due misure è decisiva per un modello profondo con normalizzazioni interne, ma non abbiamo evidenza a favore di

una scelta introdotta per convinzione — e questo va detto.

VERIFICA E4 — ipotesi sullo scarto dei FLOPs formulata e confutata

Ipotesi del 17/08: il paper scrive «convolutional layers» al plurale, quindi lo scarto dell'11 % sul costo viene da strati convolutivi aggiuntivi. Testata aggiungendo strati 3×3 a stride 1:

```
1 conv: 0,574 GFLOP   (-11,0%)
2 conv: 1,703 GFLOP   (+164,0%)
3 conv: 2,831 GFLOP   (+339,0%)
```

Confutata, e nettamente. Una conv 3×3 con 196 canali alla risoluzione piena costa da sola più di 1 GFLOP: aggiungere strati non colma lo scarto, lo travolge. Lo scarto dell'11 % resta **non spiegato** e verrà riportato come tale.

VERIFICA E5 — il numero di teste è gratuito

Con $d=224$, le configurazioni a 4, 7, 8 e 14 teste hanno **parametri e FLOPs identici** (4 899 151 e 0,5742 G). La scelta è libera: si adotta $h=7$, che dà dimensione per testa 32, il valore convenzionale in letteratura, invece dei 28 di $h=8$.

ARTEFATTO Manuale_tecnico.pdf

Nuovo documento: manuale tecnico completo in 12 capitoli — matematica (STFT, scala mel, attenzione, complessità, Adam/AdamW, cross-entropy, interpolazione bilineare), architettura dei due modelli, metodo di ricostruzione dai vincoli numerici, catalogo delle assunzioni con lo stato dell'evidenza, documentazione del codice file per file, protocollo sperimentale.

Tre marcatori distinguono *dal paper / assunzione / evidenza*, così a colpo d'occhio si vede cosa è dichiarato dagli autori, cosa abbiamo deciso noi e cosa abbiamo misurato. Documento vivo, aggiornato come il diario.

DECISIONE Configurazione adottata per il dense-transformer

```
c196_proj96 · stem paper · pool (2,1) · pool_freq · d=224 · L=8 · h=7
4.899.147 parametri   (+2,1% sui 4,80 M dichiarati)
0,574 GFLOP           (-11,0% sugli 0,645 dichiarati)
```

Si addestra anche la variante `flatten`, che fitta meglio il costo (-5,9 %) e peggiora i parametri: le due bracketizzano l'incertezza. `pool_freq` resta la scelta meno sicura, perché media via la struttura in frequenza proprio nel baseline che verrà confrontato con un modello che campiona nel piano tempo-frequenza.

Ci fermiamo qui deliberatamente: nessuna configurazione soddisfa entrambi i vincoli, e continuare ad aggiungere gradi di libertà finché i numeri combaciano sarebbe overfitting su una cifra riportata senza convenzione dichiarata.

DECISIONE Chiarito l'obiettivo: riprodurre, non migliorare

Verificato sulle fonti. Pagina del corso: «you will be asked to work at home to **reproduce some (simplified) experimental results**». Testo dell'assegnazione: «riprodurre i risultati sperimentali, **reimplementando il metodo di apprendimento in modo autonomo**», con la semplificazione autorizzata riferita alla *scala* (dataset, dimensione del modello).

Ne segue la regola operativa: **il default è la lettura letterale del paper; le nostre aggiunte sono varianti opt-in valutate come ablation**. Non il contrario, com'era finora. Distinzione introdotta fra *colmatore* (il paper tace, qualcosa dobbiamo mettere) e *deviazioni* (il paper descrive e noi facciamo diverso): solo le seconde sono un problema.

VERIFICA Cercate le repository dei paper

Paper	Repo
Kavaki & Mandel 2025	nessuna — cercato in tutte e 5 le pagine, nessun link né menzione
EAT	<code>github.com/Alibaba-MIIL/AudioClassification</code> , dichiarata nell'abstract
SparseFormer	<code>github.com/showlab/sparseformer</code> , «code will be publicly available at»

Nota: il grep su SparseFormer non trovava nulla — ripgrep aveva classificato il testo estratto come binario per via dei simboli matematici. Il link si è trovato leggendo la prima pagina. Piccolo promemoria a non fidarsi di un «nessun risultato».

Attenzione alla versione: la repo di SparseFormer contiene anche una libreria corrispondente alle versioni ICLR 2024 e CVPR 2024, successive. Il file giusto è `imagenet/models/sparseformer.py` , la v1 dell'arXiv 2304.03768 che Kavaki & Mandel citano.

ERRORE La mia PhaseMix era sbagliata su cinque dettagli su cinque

L'avevo dedotta dalla frase di EAT «mixes phase components while preserving amplitude». Il codice ufficiale dice altro su ogni punto:

	EAT (reale)	La nostra versione
Trasformata	STFT , finestra di Hann	FFT dell'intero segnale
Ampiezza	preservata , non mescolata	mescolata linearmente
Fase	interpolata sugli angoli	sui vettori unitari
λ	Uniform[0,1]	Beta(0,2 · 0,2)
Peso etichetta	$\lambda \cdot 0,5 + 0,5$	λ

«Preserving amplitude» significava letteralmente *non mescolare affatto l'ampiezza*: si prende quella di un campione e solo la fase dell'altro. E il peso dell'etichetta che non scende mai sotto 0,5 — perché mescolare la sola fase altera il segnale meno — è una finezza che nessuna descrizione testuale poteva suggerire.

Lezione: dedurre un metodo da una riga di prosa è un esercizio di immaginazione, non di riproduzione. Dove esiste codice di riferimento, va letto.

ERRORE Anche `mixing` non era quello che pensavo

Non è il mixup di Zhang et al. ma il **between-class learning** di Tokozume et al. 2017 — che EAT cita come [14] e che avevamo in bibliografia senza collegarlo. I due segnali sono bilanciati in base al loro **livello sonoro** in dB e il risultato è rinormalizzato in energia:

```
G = 10 log10 E[x²]
p = 1/(1 + 10^((G1-G2)/20)) · (1-λ)/λ
x = (p·x1 + (1-p)·x2) / sqrt(p² + (1-p)²)      etichette pesate con λ, non con p
```

Il peso dei *dati* è p, corretto per il livello; quello delle *etichette* è λ, il rapporto percettivo voluto. La rinormalizzazione impedisce che il volume della miscela diventi un indizio dell'augmentation.

CORREZIONE Lo stem: niente BatchNorm, ma LayerNorm dopo il pooling

Il codice di SparseFormer conferma la descrizione del paper e la completa:

```
conv 7x7 stride 2 pad 3 (bias=True) -> ReLU -> maxpool 3x3 -> LayerNorm sui canali
```

La nostra `BatchNorm2d` fra conv e ReLU era una deviazione su **tre punti**: tipo di normalizzazione, posizione, e `bias=False` sulla convoluzione. Corretto, con test di regressione che verifica la sequenza esatta dei moduli e l'assenza di `BatchNorm`.

La differenza non è cosmetica: la `BatchNorm` normalizza ogni canale rispetto al batch, con statistiche da congelare in valutazione; la `LayerNorm` sui canali normalizza ogni posizione rispetto ai propri canali, è indipendente dal batch e si comporta identicamente in training e inferenza.

VERIFICA Risolte due ambiguità del metodo sparso

«**Normalization by three standard deviations**», che era una delle nostre assunzioni aperte, ha una formula precisa:

```
offset = (offset - offset.mean(-2)) / (3 * (offset.std(-2) + 1e-7))
```

Media e deviazione sull'asse dei *P punti*, non sul batch. Dopo la standardizzazione la deviazione è $1/3$, quindi per una distribuzione quasi normale il 99,7 % della massa cade dentro la regione: la regola dei tre sigma applicata per costruzione.

L'inizializzazione: token da normale troncata con std 1, regioni su una griglia `root × root` con `root =  $\sqrt{N}$` in coordinate normalizzate. Da qui una scoperta collaterale: **N deve essere un quadrato perfetto**, ed è esattamente ciò che si osserva nella Tabella 3 del paper, la cui ablation usa $N \in \{4, 9, 16, 25, 36\}$. Non era una scelta estetica degli autori ma un vincolo strutturale ereditato — un dettaglio che ora sappiamo spiegare.

VERIFICA Un test fallito che ha insegnato qualcosa: la consistenza della STFT

Avevo scritto un test che verificava «phasemix preserva il modulo dello spettrogramma». Fallito: errore relativo del **34 %**, non dello 0.

Non era un bug. Una matrice complessa arbitraria **non è la STFT di alcun segnale reale**: con finestre sovrapposte al 60 % i frame adiacenti condividono campioni e si vincolano a vicenda, e alterare le fasi rende la matrice *inconsistente*. La iSTFT fa overlap-add, che equivale a proiettare sul sottospazio delle STFT consistenti — e la proiezione cambia anche i moduli.

«Ampiezza preservata» vale quindi sulla *matrice modificata*, non sul segnale ricostruito. È plausibilmente parte di ciò che rende efficace l'augmentation: il risultato non è una semplice ricombinazione dei due originali. Stesso fenomeno che rende necessario Griffin-Lim per ricostruire da spettrogrammi di sola ampiezza.

Il test è stato riscritto in due: con $\lambda=1$ phasemix è l'identità entro 10^{-4} (correttezza del percorso STFT→iSTFT), e con $\lambda=0$ la deviazione dei moduli è misurabile ma limitata. Suite ora a **26 test**.

CORREZIONE Normalizzazione dello spettrogramma disattivata di default

Coerente con la regola stabilita oggi. All'evidenza contraria della sonda lineare si aggiunge ora un argomento più forte: **né il paper né i due ancestor**, di cui abbiamo letto il codice, normalizzano lo spettrogramma. Resta come flag e ablation dichiarata.

Verificato che la configurazione adottata regge dopo tutte le correzioni: 4 899 147 parametri (+2,1 %), 0,574 GFLOP (−11,0 %), sequenza di 51 frame.

VERIFICA Audit completo prima del training

Eseguito un audit che esercita la catena intera su dati veri, non i singoli pezzi:

```

forward completo      logits (64,35) finiti
backward              gradiente su tutti i 108 tensori, nessuno identicamente nullo
train/eval            delta esattamente 0.00e+00
EMA                   passo 0.0050 = 1 - decay, forward funzionante
augmentation con lam forzato  tutti i percorsi

```

Il delta `0.00e+00` fra train ed eval conferma che la sostituzione BatchNorm → LayerNorm ha eliminato **ogni dipendenza dal batch**. Prima quel numero sarebbe stato diverso da zero e il modello si sarebbe comportato diversamente in valutazione.

Un solo problema trovato: mancava `configs/dense.yaml`, cioè la configurazione adottata viveva solo nella prosa dei documenti. Creato, con ogni scelta annotata come *dichiarata*, *dedotta* o *assunta*, più una guardia di regressione che fallisce se una modifica sposta i numeri fuori dal budget del paper.

VERIFICA Il modello sparso come secondo vincolo: predizione a -0,8 %

Calcolato il budget di parametri del modello sparso componente per componente, senza implementarlo, per sciogliere un'ambiguità rimasta: «the final decoded output $x^{(2)}$ is applied to a linear layer with dimension d » — ma $x^{(2)}$ ha forma $[P, C]$, quindi il layer opera sul tensore appiattito o su una riduzione?

```

x(2) appiattito: Linear(P·C -> d)    2.846.623    -0,8%
ridotto sui punti: Linear(C -> d)    2.201.503    -23,3%
dichiarato: 2,87 M

```

Ambiguità risolta senza margine: è il tensore appiattito. Ma il risultato vale soprattutto per un'altra ragione: **abbiamo predetto il conteggio dei parametri di un modello che non abbiamo ancora scritto, entro lo 0,8 %**. È l'evidenza più forte che la nostra lettura del metodo sia corretta — più di qualunque test unitario.

CORREZIONE Il mel è giusto, ma non per la ragione che credevamo

Verificato sulle fonti: **EAT non usa il mel**. È l'opposto — opera sulla forma d'onda grezza e argomenta che «the use of mel-spectrogram comes at the cost of having to carefully adjust the parameters for time-frequency resolution and compression».

Il sostegno alla scelta viene da altrove: **KWT** usa «Mel-spectrogram patches» (per parola dello stesso Kavaki), **AST** usa spettrogrammi con banco mel, ed EAT stesso riporta che i sistemi audio si basano «mostly on a mel-spectrogram representation», citando una comparazione sistematica che ne deduce la superiorità.

Conclusione invariata, motivazione corretta. All'orale un'affermazione giusta con la motivazione sbagliata è peggio di un dubbio dichiarato.

VERIFICA Letti `trainer.py` e `helper_funcs.py` di EAT: tre punti chiusi

Programmazione delle augmentation. `BatchAugs.__call__`: si mescola se `rand() <= mix_ratio` e `epoch > epoch_mix`; poi si sceglie una augmentation a caso e la si applica all'intero batch. Il default `mix_ratio=1` significa **sempre** — noi avevamo assunto 0,5.

One-cycle. `pct_start=0.1` esplicito; `div_factor` e `final_div_factor` ai default PyTorch (25 e 10^4), quindi lr iniziale $1,2 \cdot 10^{-5}$ e finale $1,2 \cdot 10^{-9}$.

Weight decay. Qui una **riabilitazione**: avevamo classificato come nostra deviazione l'esclusione di bias e normalizzazioni. Ma EAT costruisce AdamW con `weight_decay=0` e delega a gruppi di parametri con criterio `len(param.shape) == 1` — che cattura bias, LayerNorm e BatchNorm insieme. Non era una deviazione: era conforme al riferimento.

ERRORE Il target mixato non è morbido: è multi-hot con soglia

L'ultima e più sottile delle correzioni. Il ramo `'bce'` di `mix_loss`:

```
target_shuffled = F.one_hot(target_shuffled, n).float() * (lam < 0.9)
one_h_mix       = torch.clamp(target + target_shuffled, max=1)
```

λ **non è un peso, è una soglia**. Non compare nel valore del target: decide solo se includere o no la seconda etichetta. Se $\lambda \geq 0,9$ il secondo suono è troppo debole e la sua classe viene scartata. E il target è **multi-hot** — entrambe le classi a 1, non λ e $1-\lambda$.

È la formulazione naturale per la BCE, che tratta ogni classe come domanda binaria indipendente: per una miscela udibile di due parole la risposta è sì a entrambe.

Noi avevamo implementato la semantica del ramo `'ce'` — dove $\lambda L_1 + (1-\lambda)L_2$ è equivalente a un target morbido per linearità. Non un errore di matematica, un errore di scelta del ramo di riferimento. Visibile solo leggendo il codice.

VERIFICA Due test falliti, entrambi colpa mia, entrambi istruttivi

Dopo il passaggio ai target multi-hot, due test asserivano che tutti i campioni mixati avessero due classi attive. Falliti su 1 campione su 16.

Causa: `randperm` ha **punti fissi**. Con probabilità $\approx 1-1/e$ almeno un campione viene mixato con sé stesso, e il `clamp` riporta quel target a una sola classe. È il comportamento del riferimento, non un difetto — circa $1/B$ dei campioni per batch riceve un'augmentation che è di fatto un no-op.

Test resi robusti e il fenomeno documentato. Suite ora a **31 test**.

DECISIONE Stato finale delle ambiguità prima del training

Resta **una sola assunzione** senza appiglio in nessuna fonte: il **numero di epoche**. Tutto il resto è dichiarato dal paper, documentato nel codice di riferimento, o supportato da una nostra misura.

Restano invece aperte quattro *questioni*, che non sono assunzioni nascoste ma incertezze dichiarate: lo scarto dell'11 % sui FLOPs (non spiegato, due candidati eliminati), `pool_freq` contro `flatten` (indecidibile dai vincoli, si addestrano entrambe), pre-norm contro post-norm (da ablare), e la verifica dei FLOPs dello sparso come terzo vincolo (rimandata alla Fase 4, perché i parametri non testano `pool_stride` mentre i FLOPs sì).

VERIFICA Controllo generale: lint, coerenza fra documenti, requirements

Un controllo diverso dagli audit precedenti, sulla coerenza fra codice, documenti e configurazioni. Tre risultati:

- **13 problemi di lint** (import non idiomatici, `__all__` non ordinati, `noqa` inutili). Corretti; i test continuano a passare, quindi le riscritture degli import non hanno rotto nulla;
- **un numero obsoleto nei documenti**: 4.899.151 contro il valore reale 4.899.147. La differenza di 4 viene proprio dalla correzione dello stem — conv con bias (+196), BatchNorm rimossa (−392), LayerNorm aggiunta (+192);
- **il documento Pipeline era fermo alla v2.0** e la sua §3.4 elencava come «da risolvere» ambiguità nel frattempo chiuse. Con tre documenti vivi la divergenza è strutturale: ho riscritto §3.4 come riepilogo di stato e stabilito una **regola di precedenza** — in caso di discordanza fa fede il manuale.

Generato anche `requirements.lock.txt` (144 pacchetti), che il nostro stesso README indicava come parte della consegna e mancava.

ARTEFATTO Fase 3: `src/train.py` e `scripts/evaluate.py`

Training loop completo: AdamW con gruppi di parametri, one-cycle coi parametri di EAT, EMA, BCE su target multi-hot, augmentation sul batch in GPU. Ogni run è determinato dal solo file di configurazione più il seed, e la config viene archiviata accanto ai risultati.

Aggiunta a `utils.py` la risoluzione della chiave `defaults:`, così `dense.yaml` eredita da `base.yaml` e mostra solo ciò che cambia.

I checkpoint si salvano in modo **atomico** (file temporaneo più rinomina): su un portatile un'interruzione per surriscaldamento non deve lasciare un checkpoint corrotto. `--resume` ripristina anche ottimizzatore e scheduler.

La guardia sul test set non è un promemoria ma un meccanismo: `evaluate.py` scrive `TEST_EVALUATED.json` al primo utilizzo e si rifiuta di ripartire. Con `--force` si può insistere, ma l'evento finisce nello storico del file. Verificato: seconda invocazione bloccata.

VERIFICA Primo training reale, e il budget va rivisto

```
epoca 1/2  loss 0.2040  val(EMA) 39,69%  val(raw) 41,41%  84s
epoca 2/2  loss 0.1616  val(EMA) 62,98%  val(raw) 63,30%  73s
```

Il modello impara. Ma **~80 s per epoca contro i 24 stimati** dal benchmark, e picco di memoria 2,7 GB contro 266 MB. Il benchmark misurava il solo encoder su dati sintetici; mancavano quattro cose: la sequenza reale è di **51** frame e non 26 (la geometria del pooling l'avevamo dedotta dopo), il calcolo del log-mel a ogni batch, le augmentation — che per phasemix includono STFT e iSTFT complete — e **due** passate di validation per epoca.

Budget rivisto: ~2,2 ore per run da 100 epoche, quindi circa **13 ore** per i sei run della Fase 3 invece delle 4 stimate. Sostenibile distribuito su più serate, ma è bene saperlo prima di lanciare.

ERRORE Doppia codifica: 800 righe di documentazione corrotte

Per correggere il numero obsoleto avevo usato una sostituzione veloce in PowerShell:

```
(Get-Content file -Raw) -replace 'a','b' | Set-Content file -Encoding UTF8
```

Sembra innocua e non lo è. In PowerShell 5.1 `Get-Content` senza `-Encoding` legge un file UTF-8 *privo di BOM* usando il codepage ANSI: ogni carattere non ASCII viene interpretato come i suoi byte separati. `Set-Content -Encoding UTF8` li riscrive poi come UTF-8, **codificandoli due volte**. Risultato: `capacità` → `capaciť`, per **475 occorrenze nel manuale e 325 nel diario**.

Scoperto per caso: un'operazione di modifica sul manuale non trovava più il testo che cercava. Senza quel fallimento la corruzione sarebbe passata inosservata fino alla lettura dei PDF.

Riparazione. L'inversa è `encode('cp1252')` poi `decode('utf-8')`, ma con due complicazioni. Primo, i file erano *misti*: alcune righe erano state riscritte correttamente dopo la corruzione e non andavano toccate — risolto lavorando riga per riga e riparando solo quelle con marcatori di mojibake. Secondo, le posizioni `0x81`, `0x8D`, `0x8F`, `0x90`, `0x9D` sono *undefined* in cp1252: il codec di Python le rifiuta mentre .NET le mappa, e senza gestirle a mano restavano irreparabili tutte le righe con frecce, esponenti o lettere greche. Recuperate tutte e 800 le righe.

Lezione: per modificare file di testo non si usano round-trip di PowerShell. Si usa uno strumento che dichiara la codifica esplicitamente, o Python.

ERRORE Il checkpointing non salvava lo stato dei generatori casuali

Verificando il checkpointing su richiesta, ho trovato che salvava modello, EMA, ottimizzatore e scheduler ma **non lo stato dei generatori casuali**. Conseguenza: un run ripreso riparte con generatori reinizializzati, quindi ordine dello shuffle, permutazioni delle augmentation e valori di λ sono diversi da quelli che il run avrebbe avuto proseguendo.

Il risultato resta valido, ma un run interrotto non è più confrontabile bit a bit con uno mai interrotto — e su un portatile le interruzioni sono la norma. Ora si salvano `python`, `numpy`, `torch` e `cuda`, e il resume lo dichiara

esplicitamente. Verificato.

ARTEFATTO Tracciamento: CSV, TensorBoard e Weights & Biases

`src/tracking.py` mette le tre destinazioni dietro una sola interfaccia, con **degrado controllato**: se W&B manca, non c'è login o cade la rete, il run prosegue e scrive comunque CSV e TensorBoard. Un esperimento da tre ore non deve morire perché un servizio esterno non risponde.

L' `id` del run W&B si conserva in `wandb_id.txt`, così `--resume` riprende la stessa curva invece di aprirne una nuova. Verificato che funziona sia online sia in `mode: offline`, che scrive in locale e si sincronizza dopo.

Aggiunti anche: **norma globale del gradiente** per epoca (costa nulla, e una norma che esplode o collassa segnala il problema prima della loss), barra di avanzamento tqdm disattivata automaticamente quando l'output non è un terminale, e `scripts/run_seeds.py` che esegue più seed, salta quelli già completati e aggrega media e deviazione standard.

ARTEFATTO Metriche per classe e confusioni

`src/metrics.py`: accuratezza per classe, accuratezza **bilanciata** (media delle accuratezze per classe, che con 2,6× di sbilanciamento può divergere da quella globale), classi peggiori e coppie più confuse. Prodotto a fine training su validation, e su test in `evaluate.py`.

Scelta deliberata: **non** una mappa di calore 35×35, che a quella dimensione è quasi illeggibile, ma l'elenco delle coppie. Già dopo una singola epoca di prova le confusioni osservate erano `seven → six`, `forward → four`, `go → down`: errori fonologicamente sensati, che sono un segnale che la catena funziona.

ERRORE Ipotesi sulla precisione mista: confutata dalla misura

Avevo proposto l'autocast **bf16** come «potenzialmente il più utile» degli strumenti da aggiungere, stimando un taglio del budget da 13 a 8-9 ore. Implementato e misurato su un'epoca completa:

```
fp32: 101,7 s/epoca  val 31,81%  loss 0,2015
bf16: 101,7 s/epoca  val 31,91%  loss 0,2015
```

Nessun guadagno. Verificato che non fosse un errore di configurazione: il `resolved_config.json` registra `amp: bf16` e il contesto di autocast, testato a parte, produce tensori `bf16`.

La spiegazione è la stessa già emersa due volte: il modello è **limitato dal lancio dei kernel**, non dall'aritmetica. A queste dimensioni le matrici sono troppo piccole perché i tensor core continuo, e il tempo se lo prendono l'overhead CUDA e le parti non-matmul — mel, STFT e iSTFT di phasemix, EMA, norma del gradiente. Che le due epoche coincidano a 0,1 s di distanza *conferma* la tesi: il tempo dipende dal numero di lanci, identico nei due casi.

Terza misura indipendente che converge sulla stessa spiegazione, dopo il confronto profondità/larghezza e i FLOPs che non predicono la latenza. `amp` resta `none`: il codice è pronto e testato, ma attivarlo aggiungerebbe una variabile numerica in cambio di nulla.

ERRORE Il resume non era riproducibile: tre bug in fila

Verifica pre-lancio: un run interrotto e ripreso dava numeri diversi da uno continuo. Prima ipotesi — rumore della GPU — **smentita**: due run identici in modalità deterministica sono bit-identici, mentre il resume divergeva di 0,32 punti.

Causa radice: **ripristinare lo stato dei generatori non basta se il numero di consumi differisce**. Il DataLoader estrae dal generatore globale per il seed del sampler e per il `base_seed` dei worker, e con `persistent_workers` lo fa una volta per *processo*. Un run ripreso ripete quelle estrazioni e sfasa tutto ciò che viene dopo.

Tre manifestazioni, corrette una alla volta: l'**ordine dei dati** (risolto con un sampler la cui permutazione è funzione pura di seed ed epoca), la **scelta dell'augmentation** (risolta riseminando a ogni epoca), e infine la scoperta che quella risemina era **tre righe troppo in alto** — va fatta dopo aver creato l'iteratore, non prima.

Verifica finale: 0 tensori diversi su 108, metriche identiche a otto decimali. Il resume è ora esattamente trasparente.

Come è stata trovata: non rileggendo il codice — che sembra corretto in tutti e tre i casi — ma confrontando due percorsi che dovevano dare lo stesso risultato. E la localizzazione è arrivata solo cambiando metrica: i pesi dopo la prima epoca ripresa invece dell'accuratezza dopo due.

DECISIONE Si addestra con `--deterministic`

Misurato: due run identici differiscono di 0,02 punti in modalità normale e sono bit-identici con `--deterministic`, al costo di **+10 %** sul tempo. Su 17 ore sono 1,7 ore in più.

In cambio: ogni numero è esattamente riproducibile, il pavimento di rumore va a zero, e l'unica varianza che resta è quella fra seed — che è ciò che vogliamo misurare. Con differenze fra configurazioni che nel paper valgono 0,21 punti, non è un dettaglio.

ERRORE Il determinismo era un avviso, non una garanzia

Primo lancio vero, fermato dopo un'epoca. Nel log, per `matmul`, `einsum`, `Linear` e tutto il backward:

```
this operation is not deterministic because it uses CuBLAS ...
you must set CUBLAS_WORKSPACE_CONFIG=:4096:8
```

Usavamo `warn_only=True`: le operazioni non deterministiche giravano comunque, segnalandolo con righe di log che scorrono via. Avremmo addestrato 2,5 ore con una falla nota nella garanzia che avevamo passato la mattina a costruire.

Correzioni: `CUBLAS_WORKSPACE_CONFIG=:4096:8` impostata in `src/__init__.py` — il primo file eseguito, l'unico posto in cui abbia effetto, e dove non può essere dimenticata da riga di comando — e `warn_only=False`, così un'operazione non deterministica diventa un errore. Il run parte lo stesso: nessuna operazione del nostro modello è in quella condizione.

Esito: 0 tensori diversi su 108, e i numeri *identici* a prima della correzione. Il determinismo osservato era reale ma reggeva per fortuna; ora regge per costruzione. Picco di memoria sceso da 2715 a 1087 MB.

Come è emerso: non da un test — i test passavano — ma leggendo i log del primo lancio. La falla era nella garanzia, non nel comportamento.

CORREZIONE Due contorni sistemati nella stessa sosta

ALBERO SPORCO era un falso positivo. `provenance()` chiama `git status` dopo che il run ha già scritto `config.yaml` e `resolved_config.json` nella propria cartella, e li contava come modifiche. Ora esclude `results/`. Un flag di riproducibilità che dà falsi positivi è peggio che non averlo.

W&B passato a `mode: offline`, per disaccoppiare la scelta dell'account dal lancio: i run restano in locale e si sincronizzano dopo, sull'account attivo in quel momento, con `wandb sync results/<tag>/wandb/offline-*`. Le curve dal vivo restano su TensorBoard, che non dipende da alcun account.

VERIFICA Tracciamento W&B online, verificato end-to-end

Risolto il problema di accesso — era una questione di sessione del browser, non di credenziali: la CLI era già autenticata come `alessandro-simone001` sull'entity `alessandro-simone001-universit-di-firenze`, dove esiste già il progetto di un altro corso.

Riportato `mode: online`. `entity` resta `null` di proposito: fissarla renderebbe il repository non eseguibile da altri, e la consegna prevede che il docente possa eseguirlo. La portabilità ha la precedenza sull'esplicitezza; l'entity effettiva finisce comunque in ogni `summary.json`.

Verificato interrogando l'API che il summary arrivi al server completo:

```
best_val_accuracy = 0.2747    gflops = 0.574231616
provenance = commit 9e314e62, dirty=True
```

Due conferme in un colpo solo. L'accuratezza è **identica** a quella del run precedente alla correzione su cuBLAS, quindi il determinismo regge. E `dirty=True` è un *vero* positivo — la modifica alla configurazione non era ancora committata: dopo la correzione di ieri il flag non mente più.

Giorno 4 — mercoledì 19 agosto 2026

Primo run completo del dense-transformer.

VERIFICA Riconciliazione: il repository era avanti rispetto al mio contesto

Riprendendo il lavoro ho trovato l'albero **pulito** ma tre commit oltre quelli che avevo in memoria: `f7b73c6` (riproducibilità del resume, *tre* bug corretti), `9e314e6` (determinismo con `CUBLAS_WORKSPACE_CONFIG`) ed `e610690` (W&B online verificato end-to-end).

La verifica pre-lancio che avevo avviato era in realtà **arrivata in fondo** — risultava «interrotta» solo perché il processo era uscito senza lasciare il record — e aveva trovato ciò che cercava:

```
ininterrotto : loss 0.135670 val 79.922%
ripreso      : loss 0.135585 val 79.752%
-> resume NON trasparente: un'interruzione cambia il risultato
```

Quei tre commit chiudono esattamente quel problema. Le modifiche che avevo in corso vi sono confluite: nessun lavoro duplicato da scartare.

ERRORE Il flag `dirty` della provenienza dava falsi positivi

Il primo run completo, lanciato da un albero senza alcuna modifica al codice, ha registrato `dirty: true`. Causa: `git status --porcelain` conta anche i **file non tracciati**, e nella radice c'erano `results/` e uno script di appunti.

Una precedente correzione escludeva `results/`, ma non bastava: qualunque file di prova nella cartella avrebbe riprodotto il problema. **Un flag di reproducibilità che grida al lupo è peggio che non averlo**, perché smetti di guardarlo proprio quando conta.

Corretto con `--untracked-files=no`: `dirty` riflette ora solo le modifiche ai file *tracciati*, che sono le uniche a cambiare il codice eseguito. I file non tracciati si contano a parte in `untracked`, così l'informazione non viene nascosta ma nemmeno confusa con l'altra.

VERIFICA Primo run completo: 94,67 %

100 epoche, **1,80 ore** a 65 s per epoca, picco GPU 1 087 MB. Il run è stato spezzato in due sessioni con `--stop-after 50` e poi `--resume`, e ha completato senza problemi.

validation (EMA)	94,67 %	parametri	4.899.147
bilanciata	94,31 %	costo	0,5742 GFLOP
paper (test)	96,67 %	scarto	-2,00 punti

Il confronto non è diretto — il nostro numero è su *validation*, il loro su *test*, che non abbiamo ancora toccato — ma siamo al limite del criterio di successo pieno che ci eravamo dati («entro 1-2 punti»).

Stima dei tempi finalmente esatta: 65 s/epoca contro i 110 previsti. I ~100 s dei run di prova erano avvio dei worker e cache fredda ammortizzati su una o due epoche soltanto.

VERIFICA La curva satura a un terzo del budget — la scoperta che conta

```
ep 30: 93,45 %
ep 50: 94,06 %      +0,61 dalla 30
ep 70: 94,49 %
ep 100: 94,67 %     +1,22 dalla 30, in 70 epoche
ultime 10 epoche: escursione 0,06 punti
```

«**Poche epoche**» è eliminato come spiegazione dei due punti mancanti. Il modello smette di migliorare molto prima della fine: il residuo è architetturale o configurativo.

Conseguenza pratica notevole per la Fase 5: **50 epoche darebbero le stesse conclusioni di 100**. Con 27 run previsti fra le ablation su N e su P, è la differenza fra ~50 e ~25 ore di GPU.

VERIFICA L'EMA si guadagna lo spazio solo ad alto learning rate

```
ep 10: +4,29 punti      lr al massimo
ep 30: +1,61
ep 50: +0,45
ep 90: +0,01
ep 100: 0,00           lr = 1,2e-09
```

Esattamente ciò che la teoria prevede: la media mobile smorza l'oscillazione dovuta al rumore dei minibatch, e quando il learning rate si annulla i pesi smettono di muoversi — la media di una traiettoria ferma è il punto stesso.

Nota operativa: con one-cycle portato a termine, valutare EMA o pesi correnti è indifferente. L'EMA conta se si interrompe prima. È un'assicurazione che non è servita, ma che costava nulla.

ERRORE Avevo letto male lo sparkline: la norma del gradiente non cresce

Dallo sparkline di W&B avevo concluso che `train_grad_norm` crescesse lungo il training, e l'avevo pure commentato come «osservazione da avere pronta». I valori dal CSV dicono altro: 0,26 all'epoca 1, picco 0,92 alla 5 mentre il learning rate sale, poi **piatta a ~0,24 dall'epoca 20 alla fine**.

Lo sparkline è binnato e riscalato sul proprio intervallo: con una metrica quasi costante amplifica il rumore fino a farlo sembrare una tendenza. **Per l'andamento di una metrica si legge il CSV, non la miniatura**. È esattamente il motivo per cui il CSV è la fonte di verità del progetto e non un ripiego.

VERIFICA Il profilo degli errori è fonologico

```
tree  -> three  9      three -> tree   8
forward -> four  8      go    -> down  10
no     -> go     7      off   -> up    10
```

tree ↔ *three* è la coppia minima /t/ contro /θ/, e la confusione è **simmetrica**: non un bias del modello, ma l'incapacità di distinguere una fricativa dentale sorda da un'occlusiva alveolare in mezzo secondo di audio. *forward* → *four* condivide l'attacco, *go/down/no* hanno vocali posteriori simili.

È il controllo qualitativo che vale più dell'accuratezza aggregata: un modello che sbagliasse coppie senza somiglianza acustica segnalerebbe un difetto nella pipeline dei dati. Qui il profilo è quello di un sistema che ha imparato la fonetica e si ferma dove si fermerebbe un ascoltatore umano.

DECISIONE Prima flatten, poi i seed

L'istinto sarebbe lanciare i seed 1 e 2. Sconsigliato: `pool_freq` è la scelta **meno sicura** dell'intera ricostruzione, e la variante `flatten` fitta meglio il costo dichiarato (−5,9 % contro −11,0 %).

Se `flatten` arrivasse più vicino al 96,67 %, sarebbe la ricostruzione migliore e i tre seed andrebbero spesi su quella. Fare ora i seed di `pool_freq` significa rischiare 3,6 ore su una configurazione da abbandonare. **Un run di flatten costa 1,8 ore e decide la questione**.

ARTEFATTO Estrattore sparso implementato

`src/models/sparse.py` e `sparse_model.py`: i tre passi del metodo — aggiustamento delle regioni con parametrizzazione Faster R-CNN, campionamento con offset appresi e normalizzazione a tre deviazioni

standard, decoding adattivo con M_c e M_s generati dal token — più l'assemblaggio con early conv, ponte 64→128 e 8 encoder senza codifica posizionale.

Verificato che il gradiente raggiunga **token e regioni iniziali**: se `box_init` non ricevesse gradiente verrebbe a mancare il meccanismo centrale del paper, quello per cui il modello impara *dove* guardare.

I parametri confermano la predizione fatta prima di implementare: **2 822 711 contro 2,87 M, -1,6 %**. Aggiunti 13 test sull'estrattore; suite a 43.

VERIFICA I FLOPs dello sparso risolvono la contraddizione «96 vs 196»

	DENSO par.	DENSO cost	SPARSO par.	SPARSO cost	err.max
c96	+1,6%	-18,2%	-1,6%	-11,8%	18,2%
c196_proj96	+2,1%	-11,0%	-0,8%	+73,1%	73,1%

Avevamo scelto `c196_proj96` sui soli vincoli del denso. Lo sparso la demolisce, e per una ragione strutturale: la early convolution è **l'11 % del costo del denso ma il 65 % dello sparso**, perché scartando il calcolo a valle il front-end diventa il termine dominante.

I due modelli vincolano cose diverse — il denso fissa d e L , lo sparso la lettura dei canali. Nessuno dei due da solo avrebbe sciolto l'ambiguità. «96 dimensional feature» è vero, «196 kernels» è il refuso.

Conseguenza: il primo run del denso (94,67 %) usa la lettura superata e **va rifatto con c96**, perché il confronto denso-contro-sparso richiede lo stesso front-end.

ERRORE Due difetti trovati dai test dell'estrattore

Overflow di `exp()`. Un test con delta patologici ha prodotto regioni di dimensione infinita o nulla: `exp()` in float32 esplode oltre ~88. Né il paper né SparseFormer prevedono un limite. Aggiunto `MAX_LOG_SCALE = 4` — al più 55× per ripetizione — con un test che verifica che **nel regime normale non si attivi mai**, quindi è una protezione e non una deviazione. Nel regime normale i delta stanno sotto un quarto del limite.

Il contatore di FLOPs falliva. `FlopCounterMode` sollevava un `AssertionError` opaco sul modello sparso: sotto `no_grad` il suo `ModuleTracker` non riesce ad agganciare gli hook a un `Parameter` espanso, e le regioni iniziali sono esattamente questo. Si conta con il grafo attivo.

Giorno 5 — giovedì 20 agosto 2026

Primo addestramento del modello sparso.

ERRORE Il modello sparso non parte in determinismo stretto

```
RuntimeError: grid_sampler_2d_backward_cuda does not have a deterministic implementation, but you set 'torch.use_deterministic_algorithms(True)'
```

Non un difetto nostro: `grid_sample`, il meccanismo centrale del paper, non ha un backward deterministico su CUDA. Il gradiente accumula i contributi dei P punti campionati con `atomicAdd`, e l'ordine delle somme in virgola mobile varia fra esecuzioni.

La scelta di far fallire invece di avvisare ha pagato. Avevamo messo `warn_only=False` ragionando che fosse «meglio non partire che scoprire dopo due ore che la garanzia non valeva»: con `warn_only=True` il run sarebbe partito, avrebbe stampato un avviso perso fra migliaia di righe, e avremmo creduto bit-riproducibili dei risultati che non lo erano.

Risolto rilassando a `warn_only` solo per lo sparso, deciso da `model.kind`: tutto ciò che può restare deterministico lo resta. Il livello effettivo finisce nel `summary.json`.

Per l'orale: il metodo proposto è intrinsecamente meno riproducibile del proprio baseline. Rende l'ablation a singolo seed della Tabella 3 ancora più fragile — gli autori non potevano avere run bit-riproducibili nemmeno volendo.

ARTEFATTO Diagnostica sul movimento delle regioni

Il rischio silenzioso del metodo è che le regioni non si muovano: il modello degraderebbe a campionamento fisso, si addestrerebbe comunque e produrrebbe un numero plausibile **senza che nulla nei log lo segnali**.

Aggiunte due misure gratuite per epoca: `region_init_drift` (spostamento delle regioni apprese dalla griglia) e `region_adjust_norm` (norma dei pesi dell'aggiustamento per-campione, che partono da *esattamente zero*). Entrambe nulle stampano `<-- MECCANISMO INERTE`.

VERIFICA Smoke dello sparso: il meccanismo è vivo

```
epoca 1/2  loss 0.2140  val(EMA) 35,65%  spostamento 0,0165  |W_adjust| 4,3160
epoca 2/2  loss 0.1684  val(EMA) 50,59%  spostamento 0,0165  |W_adjust| 4,3445
```

`|W_adjust|` passa da **esattamente zero a 4,32** dopo una sola epoca: il ramo di aggiustamento riceve gradiente e sta imparando. Era il rischio numero uno del progetto ed è escluso.

Lo spostamento identico fra le due epoche non è un problema: con `--epochs 2 one-cycle` comprime tutto lo schedule e alla seconda epoca il learning rate è $1,3 \cdot 10^{-9}$.

VERIFICA Il modello che costa un decimo in FLOPs è più lento del 20 %

denso	0,5275 GFLOP	65 s/epoca	1.087 MB
sparso	0,0485 GFLOP	78 s/epoca	596 MB
	-91% di FLOPs	+20% di tempo	-45% di memoria

[corretto il 22/08] I 78 s/epoca vengono da uno smoke di due epoche e sono sbagliati: a regime sono ~50. E la conclusione vale solo a batch 1 — a batch 64 lo sparso è più veloce. Vedi il Giorno 6.

È la **quarta misura indipendente** che converge sulla stessa spiegazione: il carico è limitato dal lancio dei kernel, non dall'aritmetica. L'estrattore sparso sostituisce pochi grandi prodotti matriciali con moltissime

operazioni minuscole, e ogni lancio CUDA costa più del calcolo che esegue.

Non invalida il paper — la riduzione di FLOPs è reale, e su un acceleratore per edge device il profilo di costo può essere diverso — ma **qualifica il claim in modo sostanziale**, con una misura fatta sui due modelli del paper stesso. La riduzione di memoria invece si traduce direttamente.

ARTEFATTO Manuale completato: le quattro lacune chiuse

Revisione sistematica del manuale contro il contenuto reale del progetto. Quattro lacune, di cui una seria:

- **EAT aveva un solo paragrafo**, pur essendo uno dei *due* paper assegnati. Nuovo §1.4 che lo copre per intero: la tesi end-to-end contro le rappresentazioni tempo-frequenza, l'architettura con le due varianti, il contributo vero (le augmentation), i risultati riportati, e il suo doppio ruolo nel nostro progetto — fonte della ricetta di training e riferimento citato ma non rimisurato.
- **Mancava il contesto della letteratura**, che la presentazione richiede esplicitamente. Nuovo §1.5 con KWT, AST, HTS-AT, EAT-S e SimPF: cosa fa ciascuno e dove si colloca la proposta. Il punto chiave è che tutti processano l'audio in modo denso e uniforme, e il paper rompe lo schema decidendo *a priori* dove guardare invece di ridurre a posteriori.
- **Il §10 documentava 11 file su 19**. Mancavano l'estrattore sparso, il modello sparso, le metriche, il tracciamento, gli script e — non banale — `src/__init__.py`, che non è vuoto e imposta `CUBLAS_WORKSPACE_CONFIG` prima di qualunque import di torch.
- **Numerazioni rotte**: due sezioni erano entrambe «10.10», e il capitolo 11 aveva la catena `11.3-bis / ter / quater / quinquies / sexies`, impossibile da citare. Rinumerato in 11.1–11.10.

Manuale alla v1.6.

VERIFICA Run di riferimento del denso, rifatto con `c96`: 94,09 %

Il primo run completo (94,67 %) usava `c196_proj96`, la lettura dei canali che i FLOPs dello sparso hanno poi scartato. Per un confronto denso-contro-sparso controllato il front-end deve essere lo stesso nei due modelli, quindi il denso è stato rifatto. 100 epoche, 1,61 h, determinismo **stretto**, albero pulito al commit `ebb9ff50`.

validation (EMA)	94,09 % (epoca 63 su 100)	parametri	4.875.235 (+1,6 %)
bilanciata	93,68 %	costo	0,5275 GFLOP (-18,2 %)
picco GPU	723 MB	tempo	55 s/epoca a regime

Il costo del rifiuto. Le due letture differiscono di 23 912 parametri — la conv 7×7 da 196 a 96 kernel (-5 000) e la proiezione 1×1 che sparisce (-18 912) — cioè lo **0,5 % del modello**, tutto nel front-end. Valgono **0,58 punti** di accuratezza, oltre a -33 % di memoria e -15 % di tempo.

Coerente con la tesi di Xiao et al. 2021 e con la ragione per cui il paper campiona *dopo* la early convolution: la capacità spesa nel front-end rende molto più di quella spesa nel transformer. È però un confronto a *un solo seed* per configurazione, e il nostro rumore stimato è 0,3–0,4 punti: sopra il rumore, ma non di molto. Va detto come indicazione, non come misura.

La saturazione è ancora più netta. Il massimo cade all'**epoca 63** e le ultime 37 epoche non producono nulla (il valore finale è 94,03 %, un'inezia *sotto* il massimo). Con il primo run il massimo era all'88. Conferma definitiva: le ablation della Fase 5 si fanno a 50–60 epoche.

VERIFICA Gli errori del run di riferimento: stesso profilo, un accento diverso

classi peggiori			confusioni piu' frequenti		
learn	85,94%	(128)	go	-> down	16
backward	87,58%	(153)	tree	-> three	10
forward	89,04%	(146)	down	-> go	8
no	91,38%	(406)	off	-> up	7

Profilo invariato — coppie fonologicamente vicine, classi rare in fondo — con una differenza: `go → down` passa da 10 a 16 e diventa la confusione dominante, mentre la simmetrica `down → go` resta a 8. Con un front-end più povero il modello perde soprattutto sulle vocali posteriori, che è esattamente ciò che una risoluzione in canali dimezzata rende più difficile distinguere.

DECISIONE L'ordine dei prossimi run

Il run completo del modello sparso è stato lanciato e interrotto durante l'inizializzazione di W&B: **nessuna epoca eseguita**, la cartella `results/sparse_seed0/` contiene solo le configurazioni archiviate. È il prossimo passo, e ha la precedenza su tutto il resto per una ragione di rischio: è l'unico numero che il progetto *non ha ancora*, ed è il numero che il progetto deve produrre.

Ordine deciso, con il costo stimato di ciascun passo:

1. `sparse.yaml`, **seed 0, 100 epoche** (~2,2 h a 78 s/epoca). Chiude la Fase 4 e dà il confronto centrale del paper.
2. `dense_flatten.yaml`, **seed 0** (~1,7 h). Decide l'ultima ambiguità aperta della ricostruzione del denso. Va fatto *prima* dei seed, per non spenderne tre su una configurazione da abbandonare.
3. **Seed 1 e 2** sulla configurazione densa vincente e sullo sparso (~7,5 h). Sullo sparso contano di più: è l'unico dei due che non può essere bit-riproducibile.
4. **Ablation su N e P** a 50 epoche (~10 h), poi il test set *una volta sola*.

VERIFICA Stato della suite: 43 test, tutti verdi

Dopo l'aggiunta dei 13 test sull'estrattore sparso la suite è a 43 e passa in 21 s senza toccare il dataset. Copre forme del front-end, invarianti delle augmentation, equivalenza dell'encoder con PyTorch, contatore di FLOPs, EMA, riproducibilità dei seed, geometria delle regioni e non attivazione del limite su `exp()` nel regime normale.

Giorno 6 — sabato 22 agosto 2026

Il risultato centrale del progetto, e una misura sbagliata due volte.

VERIFICA Il modello sparso: 95,57 %, e il claim del paper è riprodotto

Run completo lanciato la sera del 20/08, 100 epoche in 1,57 h, determinismo `warn_only`.

validation (EMA)	95,57 %	(epoca 97 su 100)	parametri	2.822.711	(-1,6 %)
bilanciata	95,12 %		costo	0,0485 GFLOP	(-11,8 %)
picco GPU	596 MB		paper	96,88 % su test	-> -1,31

Il confronto centrale, a parità di tutto tranne la produzione dei token:

	parametri	GFLOP	val EMA	
denso c96	4.875.235	0,5275	94,09 %	
sparso	2.822.711	0,0485	95,57 %	<-- +1,48
	(0,58x)	(0,092x)		
paper: denso	96,67 %	sparso	96,88 %	-> +0,21

[**corretto il 23/08**] Questo margine è calcolato contro `pool_freq`, che il run di `flatten` ha poi scartato come baseline. Col denso corretto il confronto si inverte: -1,44. Vedi il Giorno 7.

Con il **58 % dei parametri e il 9,2 % dei FLOPs** il modello sparso è **1,48 punti sopra** il denso: il claim qualitativo del paper è riprodotto, e con un margine sette volte più ampio del loro. Il nostro 1,48 è ben oltre il rumore da seed stimato (0,3–0,4 punti); il loro +0,21 *non lo è*.

VERIFICA Lo scarto dal paper si dimezza dove il paper è esplicito

modello	nostro	paper	scarto	quanto il paper dichiara
denso	94,09 %	96,67 %	-2,58	due numeri; tutto il resto RICOSTRUITO
sparso	95,57 %	96,88 %	-1,31	Tabella 1 completa; NIENTE da ricostruire

La riproduzione è più fedele proprio dove non abbiamo dovuto indovinare, e lo scarto raddoppia dove sì. È l'argomento più forte che il residuo del denso sia **errore di ricostruzione** e non un difetto della pipeline: dati, front-end, augmentation e ottimizzatore sono condivisi dai due modelli, quindi un problema lì darebbe due scarti simili.

Da tenere pronto all'orale: è la risposta alla domanda «avete riprodotto il paper?», e trasforma uno scarto in un'osservazione.

VERIFICA Le regioni si muovono per 70 epoche, poi si fermano

epoca	1	10	20	30	50	70	100
drift	0,0017	0,0205	0,0571	0,0889	0,1343	0,1437	0,1437
w_adj	0,42	4,91	8,93	9,79	12,84	13,28	13,42

Lo spostamento cresce di un fattore **84** e poi si ferma: identico alla quarta cifra fra l'epoca 70 e la 100. È il comportamento migliore fra quelli possibili, e si apprezza guardando i due modi in cui poteva fallire — drift a zero (meccanismo inerte, campionatore a griglia travestito) o drift senza fine (regioni alla deriva fuori dal piano, a leggere sempre il bordo). Invece **convergono**.

Spiega anche perché lo sparso impari più a lungo del denso: massimo all'epoca 97 contro 63. Finché le coordinate si muovono il modello sta ancora cambiando *quali dati riceve*, non solo come li elabora; il denso quel grado di libertà non ce l'ha.

ERRORE La misura di latenza era sbagliata. Due volte, in direzioni opposte

Il manuale riportava «sparso 78 s/epoca contro 65 del denso, +20 %». Quel numero veniva da uno **smoke test di due epoche**, che ammortizza avvio dei worker e cache fredda su due sole epoche. A regime sono **~50 s contro ~55**.

È lo stesso errore già commesso sul denso il 18/08 (110 s stimati, 65 reali), già scritto in questo diario come lezione — e ricomesso il giorno dopo averlo scritto.

Ma anche la correzione andava qualificata, e la seconda misura lo ha mostrato: il tempo per epoca è una misura a **batch 64**. Misurando il solo modello con warmup, su GPU scarica:

	GFLOP	ms @batch 1	ms @batch 64
denso c96	0,5275	2,95	11,08
sparso	0,0485	4,65	6,49
rapporto	0,092x	1,58x	0,59x
		PIU' LENTO	PIU' VELOCE

Un fattore **10,9 sui FLOPs** vale **0,63 a batch 1** e **1,71 a batch 64**: nessuno dei due è 10,9, e i due hanno segno opposto. La conclusione originale era giusta per il caso sbagliato.

Il punto scomodo per il paper, e quello da portare all'orale: il regime in cui il metodo perde è batch 1, cioè il dispositivo edge che classifica una parola alla volta — lo scenario che motiva l'intero lavoro.

Lezione, diversa da quella del 18/08 perché stavolta è diventata uno strumento: il tempo per epoca non è latenza, e una stima da una o due epoche misura l'avvio. Da qui `scripts/benchmark_models.py`, che misura sul solo modello, con warmup, su GPU scarica e a più di un batch. Una lezione scritta e non trasformata in strumento non è stata imparata.

ARTEFATTO Tre strumenti che sbloccano la Fase 5

- **Override da riga di comando** (`--set model.num_tokens=9`): le due ablation escono da `sparse.yaml` senza scrivere nove YAML quasi identici, che sarebbero divergiti al primo cambiamento della base. Due guardie: la chiave deve già esistere (un refuso farebbe girare un'ablation che usa il default, senza alcun sintomo) e il tipo deve combaciare. Senza `--tag` si rifiuta di partire, per non sovrascrivere la cartella del run di base.
- **Coordinate del campionamento nel trace**. `sampling_trace()` restituiva regioni e token ma *non* i punti: `SparseSampler.forward` calcolava `coords` e le consumava. La Figura 2 era indisegnabile e non ce n'eravamo accorti. Ora escono, con `return_coords` opt-in e un test che verifica che il percorso di training sia rimasto identico.
- `scripts/plot_sampling.py`: la Figura 2 del paper dai pesi di un run — regioni e punti colorati per token, sovrapposti allo spettrogramma, alle tre ripetizioni.
- `scripts/benchmark_models.py`: costo e latenza dei due modelli a più batch, con warmup e controllo sull'occupazione della GPU.

Suite a **48 test** (erano 43).

ERRORE Una guardia che dava un falso positivo, scritta oggi

`benchmark_models.py` avvisava «la GPU ha già 1141 MB occupati: un altro processo falserà la misura» con la GPU vuota. Causa: `torch.cuda.mem_get_info()` è a livello di dispositivo e include il contesto CUDA del processo che la chiama, che da solo vale ~1 GB.

È la terza guardia di riproducibilità del progetto che nasce con un falso positivo, dopo le due sul flag `dirty`. Il modo di sbagliare è sempre lo stesso: *misurare uno stato globale dall'interno del processo che lo ha appena modificato*. Corretta stampando il contesto come informazione e rimandando la verifica seria a `nvidia-smi`, che sta fuori.

DECISIONE **Correzione alla regola «ablation a 50 epoche»**

La regola era stata dedotta dal *solo* modello denso, che ha il massimo all'epoca 63. Sullo sparso il massimo è alla 97, e a 50 epoche siamo a 95,13 % contro 95,57 %, cioè **0,44 punti sotto**.

La regola resta, ma enunciata con precisione: **a 50 epoche l'ordinamento fra configurazioni si conserva, i valori assoluti sono ~0,4 punti più bassi**. Per un'ablation va bene; per un numero da mettere in tabella accanto a quelli del paper, no. I due usi vanno tenuti separati — ed è il tipo di distinzione che, se non scritta ora, fra un mese produce una tabella con due scale diverse.

ERRORE **La Figura 2, appena generata, trova un difetto che due diagnostiche non vedevano**

Primo bug, banale: `plot_sampling.py` caricava `results/<run>/config.yaml`, che è la copia archiviata del *sorgente* e contiene ancora `defaults: base.yaml` — file che in quella cartella non esiste. Corretto leggendo la configurazione dal checkpoint, come fa già `evaluate.py`: è anche più giusto, perché è la config *realmente* usata, override inclusi. Verificato che `evaluate.py` non avesse lo stesso difetto — usa già `state["config"]`.

Poi il ritrovamento vero. La figura stampa la geometria per stadio:

	dimensione media	massima	punti DENTRO il piano
ripetizione 1	0,54	1,28	98,6 %
ripetizione 2	2,49	9,71	50,0 %
ripetizione 3	54,88	431,22	50,0 %

Il campionamento è sano solo alla prima ripetizione. Dalla seconda le regioni escono dal piano tempo-frequenza e metà dei punti legge il bordo (`padding_mode="border"`). Nella figura si vede a occhio: alla terza i punti formano **strisce verticali**, che è la firma di una coordinata saturata al bordo mentre l'altra varia ancora.

Né `region_init_drift` né `region_adjust_norm` potevano accorgersene: la prima misura solo lo spostamento delle regioni *iniziali* (che sono a posto, 0,1437), la seconda solo la norma dei pesi. Entrambe dicevano «il meccanismo è vivo», ed era vero. Nessuna delle due poteva dire *dove* stesse andando.

Lezione. Nel manuale avevamo scritto che le diagnostiche numeriche non bastano e che serviva la figura «per vedere *dove* vanno le regioni». Era vero, ed è rimasto vero per due giorni senza che la generassimo. Una verifica rimandata è una verifica non fatta.

ERRORE **«Il clamp non si attiva mai»: dedotto da un test che non lo diceva**

ripetizione 1:	delta medio	0,38	clamp attivo sullo	0,0%
ripetizione 2:	delta medio	4,40	clamp attivo sul	45,9%
ripetizione 3:	delta medio	5468,64	clamp attivo sul	50,0% (max 43816)

Il test `test_region_clamp_never_binds_in_the_normal_regime` misurava i delta **all'inizializzazione**, dove `to_delta` è azzerato per costruzione e il limite *non può* attivarsi. Da lì avevamo dedotto una proprietà del modello **addestrato**: una deduzione che non segue, finita nel manuale come fatto.

Conseguenza sostanziale: `MAX_LOG_SCALE = 4`, dichiarato come rete di sicurezza inerte, è **l'unica ragione per cui il run non ha prodotto NaN**. Con `exp(43816)` il modello sarebbe andato a infinito. Una nostra aggiunta, non prevista dal paper, è diventata *portante*: va detto all'orale prima che lo chiedano.

Test rinominato in `test_region_clamp_does_not_bind_at_initialisation`, col docstring che dice ciò che **non** verifica e riporta i numeri misurati sul checkpoint.

VERIFICA **Alla prima ripetizione le regioni sono identiche per ogni ingresso**

Visibile solo nella figura, e non è un difetto ma una necessità algebrica: i token entrano nel primo stadio come `token_init`, che è un parametro e non dipende dall'ingresso, quindi `Linear(t)` produce lo stesso spostamento per ogni clip. Le regioni diventano dipendenti dai dati **solo dalla seconda ripetizione**.

Il che rende il difetto precedente più serio di quanto sembri: le uniche ripetizioni *adattive* sono esattamente quelle che degenerano. Ciò che sopravvive è un campionamento appreso ma **fisso**, uguale per ogni ingresso.

Previsione verificabile, ed è la ragione per cui vale la pena riportarlo: se le ripetizioni 2 e 3 sono in gran parte degeneri, `--set model.repeats=1` dovrebbe perdere poco. Un run da 1,6 h decide la questione, e sarebbe una critica al paper sostenuta da una misura invece che da un'impressione.

Giorno 7 — domenica 23 agosto 2026

Il baseline cambia, e con esso il risultato principale del progetto.

VERIFICA

flatten fa 97,01 % e chiude la ricostruzione del denso

	parametri	costo	accuratezza	
pool_freq	4.875.235	0,5275 G	94,09 %	
flatten	5.197.795	0,5604 G	97,01 %	<-- +2,92
dichiarati	4,80 M	0,645 G	96,67 %	

+2,92 punti, sette volte la soglia di decisione fissata prima di vedere il numero. E la curva è ancora più eloquente del valore finale: `flatten` raggiunge **all'epoca 5** (94,10 %) il risultato che `pool_freq` ottiene in cento.

La ricostruzione è decisa da un terzo vincolo che prima non avevamo. Su parametri e FLOPs nessuna delle due letture vinceva, e il manuale le teneva entrambe dichiarando che «bracketizzano l'incertezza». L'accuratezza dichiarata è il terzo vincolo, e diventa disponibile solo dopo aver addestrato entrambe: `flatten` la riproduce a +0,34, `pool_freq` la manca di 2,58. Due vincoli su tre, e quello che decide è il comportamento invece della taglia.

Il manuale aveva dichiarato `pool_freq` «la scelta meno sicura di tutta la ricostruzione» con la motivazione esatta: media via la frequenza proprio nel baseline che verrà confrontato con un modello che campiona nel piano tempo-frequenza. **La previsione era giusta; l'entità molto maggiore dell'atteso.**

ERRORE

Con il baseline corretto, il claim del paper non si riproduce

	marginale sparso - denso
contro pool_freq (sbagliato)	+1,48
contro flatten (corretto)	-1,44 <--
dichiarato dal paper	+0,21

Il 20–22/08 avevamo scritto, qui e nel manuale, che **il claim qualitativo era riprodotto con un margine sette volte più ampio del loro**. Era condizionato a un baseline che questo run ha scartato. Con il denso corretto il confronto si **inverte**: il modello sparso resta 1,44 punti *sotto*, mentre il paper lo dà 0,21 *sopra*.

Lezione di metodo. Il numero più importante del progetto è rimasto sbagliato per due giorni non per un errore di calcolo, ma perché poggiava su un'assunzione dichiarata come incerta e trattata come risolta. La decisione di fare `flatten` *prima* dei seed è ciò che ha impedito di spendere 6,5 ore su un baseline sbagliato — ed è la stessa regola che il 19/08 aveva evitato i tre seed su `pool_freq`. La regola vale: **prima si chiudono le ambiguità, poi si spendono i seed**.

Ne esce un progetto più forte, non più debole, e i criteri di successo lo dicevano già: *un progetto che afferma «non ho riprodotto il 96,88 %, ed ecco le cause candidate isolate con questi esperimenti» vale più di un numero identico ottenuto senza capire perché*. Ora abbiamo un baseline riprodotto a +0,34, uno scarto localizzato nell'estrattore e una causa già misurata.

VERIFICA

Ablation su L_{rep} : le ripetizioni rendono +0,59 punti

L_{rep}	parametri	GFLOP	accuratezza	punti dentro il piano
1	2.010.591	0,0349	94,98 %	100,0 %
3	2.822.711	0,0485	95,57 %	98,6 / 50,0 / 50,0 %
	+40 %	+39 %	+0,59	

La previsione era «se le ripetizioni 2 e 3 sono degeneri, `repeats=1` dovrebbe perdere poco». Confermata nella direzione, non nella forma forte: +0,59 è sopra il rumore da seed, quindi il raffinamento iterativo *fa* qualcosa. Non sono inerti, sono **dimezzate**.

E la geometria lo spiega: con una sola ripetizione **il 100 % dei punti resta dentro il piano** (dimensione media 0,748), contro il 50 % delle ripetizioni 2 e 3 nel modello completo. **La degenerazione è cumulativa** — nasce dal comporsi degli esponenziali attraverso le ripetizioni, non dal singolo passo.

Domanda aperta, e vale un run: con un vincolo che tenga le regioni dentro il piano, le due ripetizioni renderebbero più di 0,59? Se sì, parte dell'1,31 che ci separa dal paper è un difetto nostro; se no, è un limite del metodo. In entrambi i casi è un risultato.

Registro delle decisioni

Data	Decisione	Motivazione
12/08	Infrastruttura di misura prima del modello	Il claim è un rapporto accuratezza/FLOPs: un contatore scritto dopo tende a confermare il risultato atteso.
12/08	Venv dedicato, non riuso di ambienti esistenti	Riproducibilità (l'ambiente è parte del deliverable); DLA2026 appartiene a un altro corso.
12/08	Criteri di successo definiti in anticipo	Per non poterli riscrivere a posteriori in base ai risultati.
12/08	Installazione pip in due passi	--index-url sostituisce PyPI; --extra-index-url esporrebbe a dependency confusion.
17/08	Baseline = dense-transformer, non EAT-S	È il confronto controllato del paper; EAT-S è un numero citato. Riduce il lavoro e il rischio.
17/08	Split ufficiali con verifica bloccante dei conteggi	Speaker-disjoint; i tre numeri dichiarati dal paper sono un test di correttezza gratuito.
17/08	Ogni assunzione marcata [ASSUNZIONE]	Il paper tace su sei punti: vanno dichiarati, non nascosti. È materiale da presentazione.
17/08	Test prima del download del dataset	Separare i bug dell'infrastruttura da quelli del modello costa 2 secondi invece di ore.
17/08	Sorgenti dei documenti in docs/	Erano in una cartella temporanea di sessione: non ritrovabili in futuro.
17/08	Encoder reimplementato invece di nn.MultiheadAttention	La consegna chiede reimplementazione autonoma; verificata l'equivalenza bit a bit col riferimento PyTorch.
17/08	Configurazione del denso ricostruita dai vincoli numerici	Il paper non dichiara iperparametri del denso: 4,80 M e 0,645 G sono gli unici appigli. +13 min per run, in cambio di FLOPs confrontabili.
17/08	A parità di parametri si preferisce larghezza a profondità	Misurato: d=128 L=24 costa 1,85× d=224 L=8 a parità di capacità (limite sui lanci di kernel).
17/08	Testare il denso con e senza positional encoding	Il paper tace; l'invarianza a permutazioni rende la domanda sperimentale legittima e quasi gratuita.
18/08	Riprodurre, non migliorare	Verificato sulle fonti: la consegna chiede di riprodurre. Il default è la lettura letterale del paper; le nostre aggiunte sono varianti opt-in valutate come ablation, non il contrario.
18/08	Distinzione fra colature e deviazioni	Il paper tace (dobbiamo mettere qualcosa) è diverso da il paper descrive e noi facciamo altro. Solo le seconde sono un problema, e vanno eliminate.
18/08	Profondità $L = 8$, non ottimizzata	Il paper la dichiara. Ottimizzare su ciò che è noto è l'errore classico del fitting: si perdono 2 punti sui parametri e si guadagna coerenza col testo.
18/08	Dove esiste codice di riferimento, si legge	PhaseMix dedotta dalla prosa era sbagliata su 5 dettagli su 5. Dedurre un metodo da una riga di testo è immaginazione, non riproduzione.
18/08	Si addestra con --deterministic	+10 % di tempo, in cambio del pavimento di rumore a zero. Le differenze che il paper discute valgono 0,21 punti: non è un dettaglio.

Data	Decisione	Motivazione
18/08	Guardia sul test set come <i>meccanismo</i> , non come promemoria	<code>evaluate.py</code> scrive <code>TEST_EVALUATED.json</code> e si rifiuta di ripartire. Selezionare sul test non produce sintomi visibili: serve un blocco, non la buona volontà.
19/08	<code>flatten</code> prima dei seed	<code>pool_freq</code> è la scelta meno sicura della ricostruzione. Tre seed su una configurazione da abbandonare costano 3,6 h; il run che decide ne costa 1,8.
20/08	Determinismo rilassato <i>solo</i> per lo sparso, e dichiarato nel <code>summary.json</code>	<code>grid_sample</code> non ha backward deterministico su CUDA. Tutto ciò che può restare deterministico lo resta, e un run non bit-riproducibile non si spaccia per tale.
20/08	Diagnostica per epoca sul movimento delle regioni	Il rischio silenzioso è un meccanismo inerte che produce comunque un numero plausibile. Due misure gratuite lo rendono impossibile da non notare.
20/08	Il denso rifatto con c96	Il confronto denso-contro-sparso deve differire solo per la produzione dei token. Tenere il vecchio run come baseline attribuirebbe all'estrattore una differenza che viene dal front-end.
20/08	Ablation a 50–60 epoche invece di 100	Misurato due volte: il massimo cade all'epoca 63 (e all'88 nel primo run). Dimezza il budget delle ablation senza cambiare alcuna conclusione.
22/08	La latenza si misura con uno strumento dedicato, non dal tempo per epoca	Il tempo per epoca è a batch 64 e include dati, augmentation, EMA e due validazioni. Sbagliato due volte in due giorni: serviva uno strumento, non un'altra lezione scritta.
22/08	Si riportano <i>due</i> regimi di latenza, batch 1 e batch 64	Il segno del guadagno si inverte fra i due. Riportarne uno solo risponde a metà della domanda senza dire quale metà.
23/08	Il baseline denso è flatten	Terzo vincolo: riproduce l'accuratezza dichiarata a +0,34, contro i -2,58 di <code>pool_freq</code> . Due vincoli su tre, e quello che decide è il comportamento invece della taglia.
23/08	Si riporta che il claim del paper non si riproduce	-1,44 contro il +0,21 dichiarato, ben oltre il rumore. Riportarlo con la causa localizzata vale più di un numero uguale al loro.
22/08	Ablation via <code>--set</code> , non via nove file YAML	File quasi identici divergono al primo cambiamento della configurazione di base. Con l'override la base resta una sola e i valori effettivi finiscono comunque in <code>resolved_config.json</code> .

Errori e correzioni

Data	Cosa è andato storto	Esito
12/08	Ipotesi sbagliata sull'ancestor del metodo (importance-map di Mandel)	Smentita il 17/08: è SparseFormer. Era marcata «da confermare», ma resta una congettura presa per plausibile troppo a lungo.
12/08	Stima della cache: 1,7 GB	Valore corretto 2,72 GB . Corretto in PDF e codice.
12/08	Generazione PDF fallita (flag inesistente, profilo Edge bloccato)	Risolto con <code>--user-data-dir</code> dedicato; ricetta salvata in <code>scripts/build_docs.ps1</code> .
17/08	Lettura PDF impossibile (<code>poppler</code> mancante)	Aggirato con PyMuPDF dall'ambiente <code>DLA2026</code> .
17/08	Cancellazione di <code>src/</code> e <code>scripts/</code>	Recuperate dal cestino, verificate. Lezione: inizializzare git subito.

Data	Cosa è andato storto	Esito
17/08	<code>.gitignore</code> copriva <code>.venv/</code> ma l'ambiente si chiama <code>DL_F</code>	Aggiunta la riga mancante prima del primo <code>git add</code> .
17/08	<code>data/</code> in <code>gitignore</code> nascondeva anche <code>src/data/</code>	Corretto in <code>/data/</code> . Intercettato con <code>git add --dry-run</code> prima del commit.
17/08	Memmap non picklabile: <code>num_workers>0</code> in crash su Windows	Apertura pigra + <code>__getstate__</code> . Test di regressione aggiunto.
17/08	Smoke test su batch non mescolato: augmentation apparentemente rotte	Era il test a essere sbagliato. Ha però rivelato che la cache è ordinata per classe .
18/08	Stem escluso dalla griglia della prima ricerca	Rifatta su 432 configurazioni: la deduzione $N=51$ regge .
18/08	Ottimizzata la profondità L , che il paper aveva già dichiarato (8)	Vincolata a 8. L'errore classico del fitting: ottimizzare su ciò che è noto.
18/08	La normalizzazione per-campione non è supportata dai dati	La sonda lineare peggiora di 0,5 punti. Declassata da default ad ablation dichiarata.
18/08	Ipotesi «più strati convolutivi spiegano lo scarto sui FLOPs»	Confutata : 2 conv danno +164 % invece di +11 %. Scarto non spiegato, riportato come tale.
18/08	PhaseMix dedotta dal testo: sbagliata su 5 dettagli su 5	Riscritta dal codice ufficiale di EAT. Trasformata, ampiezza, fase, $\hat{I}$ e peso dell'etichetta erano tutti diversi.
18/08	<code>mixing</code> scambiato per il mixup di Zhang et al.	È il between-class learning di Tokozume et al. con correzione di potenza. Riscritto.
18/08	BatchNorm nello stem, con <code>bias=False</code> sulla conv	Deviazione su tre punti. Corretta in <code>conv(bias) → ReLU → maxpool → LayerNorm</code> sui canali.
18/08	Test «phasemix preserva il modulo» fallito al 34 %	Non era un bug: è l' inconsistenza della STFT . Test riscritto, fenomeno documentato nel manuale.
18/08	Grep su SparseFormer: «nessun risultato»	ripgrep aveva classificato il file come binario. Il link c'era. Non fidarsi di un negativo.
18/08	Target mixato implementato come morbido $\lambda y_1 + (1-\lambda)y_2$	È la semantica del ramo <code>'ce'</code> . Quello <code>'bce'</code> che usiamo fa multi-hot con soglia $\lambda < 0,9$. Corretto.
18/08	<code>mix_ratio</code> assunto a 0,5	Il default di EAT è 1 : si mescola sempre. Corretto.
18/08	Weight decay selettivo classificato come nostra deviazione	Classificazione sbagliata : EAT esclude i parametri con <code>len(shape)==1</code> . Era conforme al riferimento.
18/08	«Il mel lo usa l'altro paper»	Falso: EAT usa la forma d'onda grezza e argomenta <i>contro</i> il mel. Conclusione invariata, motivazione riscritta su KWT e AST.
18/08	Test che assumevano due classi attive in ogni target mixato	<code>randperm</code> ha punti fissi: $\sim 1/B$ dei campioni si mixa con sé stesso. Test resi robusti.
18/08	Configurazione adottata solo nella prosa dei documenti	Creato <code>configs/dense.yaml1</code> più una guardia di regressione sui budget.

Data	Cosa è andato storto	Esito
18/08	Numero di parametri obsoleto nei documenti (4.899.151)	Il valore reale è 4.899.147: la differenza viene dalla correzione dello stem. Aggiornato.
18/08	Documento Pipeline fermo alla v2.0, in contraddizione col manuale	\$3.4 riscritta come riepilogo di stato; stabilita una regola di precedenza fra i tre documenti.
18/08	Doppia codifica: 800 righe di documentazione corrotte	Round-trip PowerShell <code>Get-Content / Set-Content</code> senza <code>-Encoding</code> . Riparate tutte. Mai più round-trip di PowerShell su file di testo.
18/08	Stima del tempo per epoca sbagliata di un fattore 3,3	Il benchmark misurava il solo encoder su dati sintetici. Budget rivisto da 4 a ~13 ore per la Fase 3.
19/08	Flag <code>dirty</code> della provenienza: falsi positivi sui file non tracciati	Corretto con <code>--untracked-files=no</code> ; i non tracciati si contano a parte.
19/08	Norma del gradiente «crescente» letta dallo sparkline	Errata: dal CSV e' piatta a ~0,24 dall'epoca 20. Gli sparkline sono binnati e riscaldati.
19/08	Stima di 110 s/epoca	Reali 65 s : i run di prova pagavano avvio worker e cache fredda su 1-2 epoche.
19/08	Front-end <code>c196_proj96</code> scelto sui soli vincoli del denso	I FLOPs dello sparso lo demoliscono (+73,1 %). La lettura corretta è <code>c96</code> : il denso è stato rifatto il 20/08.
19/08	Overflow di <code>exp()</code> nell'aggiustamento delle regioni	Trovato da un test con delta patologici. Aggiunto <code>MAX_LOG_SCALE = 4</code> , con un test che verifica che nel regime normale non si attivi mai.
19/08	<code>FlopCounterMode</code> falliva sul modello sparso	Sotto <code>no_grad</code> il suo <code>ModuleTracker</code> non aggancia gli hook a un <code>Parameter</code> espanso — e le regioni iniziali sono esattamente questo. Si conta con il grafo attivo.
20/08	Il modello sparso non parte in determinismo stretto	Non un difetto nostro: <code>grid_sample</code> non ha backward deterministico su CUDA. Rilassato a <code>warn_only</code> solo per lo sparso, e il livello finisce nel <code>summary.json</code> .
20/08	Numerazione doppia nel manuale (due sezioni «10.16»)	Corretta in 10.20. Terza volta che una rinumerazione manuale sfugge: le sezioni si citano, quindi devono essere uniche.
22/08	Latenza dello sparso stimata da uno smoke di 2 epoche	78 s/epoca dichiarati, ~50 reali. Stesso errore del 18/08 sul denso, ricompresso il giorno dopo averlo scritto come lezione. Nato <code>benchmark_models.py</code> .
22/08	«Il modello sparso è più lento del 20 %»	Giusto per il caso sbagliato: è 1,58× più lento a batch 1 e 1,71× più veloce a batch 64. Il segno dipende dal regime.
22/08	<code>sampling_trace()</code> non restituiva i punti campionati	Solo regioni e token: la Figura 2 era indisegnabile. Aggiunte le coordinate, con test che il percorso di training resti identico.
22/08	Guardia sull'occupazione GPU: falso positivo a GPU vuota	<code>mem_get_info</code> include il contesto CUDA del processo chiamante (~1 GB). Terza guardia del progetto a nascere con un falso positivo, sempre per lo stesso motivo.
22/08	<code>plot_sampling.py</code> caricava <code>config.yaml</code> dalla cartella del run	Contiene ancora <code>defaults: base.yaml</code> , assente lì. Corretto leggendo dal checkpoint. <code>evaluate.py</code> verificato: non ha il difetto.
22/08	«Il clamp su <code>exp()</code> non si attiva mai»	Dedotto da un test sull'inizializzazione. Sul modello addestrato è attivo sul 46–50 % delle componenti, ed è l'unica ragione per cui il run non ha prodotto NaN.
23/08	«Il claim qualitativo è riprodotto, con margine più ampio del loro»	Poggiava su <code>poo1_freq</code> , baseline poi scartato. Col denso corretto il margine si inverte : -1,44 invece di +1,48.
23/08	<code>poo1_freq</code> adottata come configurazione del denso	Sbagliata di 2,92 punti. Era dichiarata «la scelta meno sicura», con la motivazione giusta: mediare via la frequenza.

Data	Cosa è andato storto	Esito
22/08	Il campionamento degenera dopo la prima ripetizione	Metà dei punti fuori dal piano dalla seconda in poi. Trovato generando la Figura 2; le due diagnostiche numeriche non potevano vederlo.
22/08	Regola «50 epoche bastano» dedotta dal solo denso	Sullo sparso il massimo è alla 97: a 50 epoche si perdono 0,44 punti. L'ordinamento si conserva, i valori assoluti no.

Stato al 23 agosto 2026

Fase	Contenuto	Stato
0	Lettura paper + ancestor, setup	fatto. Restano: repo Codeberg con dl.unifi , form risorse, ricevimento
1	Prerequisiti teorici	in corso — il manuale, con il capitolo 13 sull'orale
2	Dati, split, contatore FLOPs	COMPLETATA
3	Early conv + dense transformer	COMPLETATA per il seed 0: flatten 97,01 % , +0,34 sul valore dichiarato. Restano i seed 1–2
4	Sparse feature extractor	COMPLETATA per il seed 0: 95,57 % , cioè 1,44 sotto il denso. Restano i seed 1–2
4-bis	EAT-S (<i>opzionale</i>)	da fare; probabilmente da saltare
5	Ablation, seed multipli, HPO	iniziata: L_{rep} fatta. Restano N , P e i seed
6	Presentazione	il materiale c'è, incluso un risultato negativo ben localizzato. Manca il grafico accuratezza/FLOPs

CODICE: CHE COSA ESISTE E CHE COSA MANCA

Componente	Stato
Pipeline dati, front-end, augmentation	completi, testati
Modello denso e modello sparso	completi, entrambi addestrati per intero
Training, resume, tracciamento, metriche	completi; 48 test verdi
Valutazione sul test set	completa, con guardia. Mai eseguita — il test set è intatto
Override degli iperparametri (--set)	fatto il 22/08
Figura del campionamento (Fig. 2)	fatto il 22/08: <code>scripts/plot_sampling.py</code> , da eseguire sui pesi del run
Latenza e costo dei modelli	fatto il 22/08: <code>scripts/benchmark_models.py</code>
Grafico accuratezza contro FLOPs	manca. È il grafico centrale della presentazione; i dati arriveranno dalle ablation
Ablation a regioni congelate	manca. Un flag che blocchi <code>to_delta</code> risponde alla domanda più interessante da porre al paper: la saliency appresa serve, o basta la griglia?

QUESTIONI APERTE

Le voci chiuse sono barrate e restano visibili: servono a ricostruire quando e come una questione è stata risolta.

- **Aperte — operative**

- Compilare il **form per le risorse di calcolo** e fissare il **ricevimento** col docente.
- Creare il **repository privato su Codeberg** e invitare l'utente `dl.unifi`: è la modalità di consegna. Il repo oggi è solo locale.
- ~~Flag di override degli iperparametri~~ — **fatto** il 22/08 (`--set`).
- ~~Script della traccia del campionamento (Figura 2)~~ — **fatto** il 22/08, dopo aver scoperto che `sampling_trace()` non restituiva i punti. Resta da *eseguirlo* sui pesi del run.

- **Aperte — sperimentali**

- ~~Il run completo del modello sparso~~ — **fatto** il 20-22/08: **95,57 %**, +1,48 sul denso.
- **I seed 1 e 2** su `dense_flatten` e sullo sparso: senza, il -1,44 resta un'indicazione e non una misura.
- **Il vincolo sulle coordinate delle regioni**: decide se l'1,31 che ci separa dal paper sia un difetto nostro o un limite del metodo.
- ~~pool_freq contro flatten~~ — **chiusa** il 23/08 addestrando entrambe: `flatten` di 2,92 punti.
- Lo scarto sui FLOPs del denso (-18,2 % con `c96`): due ipotesi formulate, entrambe eliminate. Verrà riportato come non spiegato.
- Codifica posizionale sul denso: il paper tace, e senza il transformer è *esattamente* invariante a permutazioni. Ablation quasi gratuita.
- Pre-norm contro post-norm: da ablare, se il budget lo consente.
- La domanda più interessante da porre al paper: **la saliency appresa serve davvero, o basta l'energia?** Un'ablation con regioni fisse su griglia risponde.

- **Chiuse**

- ~~Leggere SparseFormer~~ — letto il 18/08, codice compreso.
- ~~Inizializzare il repository git~~ — fatto il 17/08, locale.
- ~~Valutare un .gitattributes con *text=auto-eol=lf~~ — fatto (commit `cf8c124`).
- ~~Risolvere la contraddizione «96 dimensional» contro «196 kernels»~~ — **risolta** il 19/08 dai FLOPs dello sparso: il 196 è un refuso.
- ~~Calibrare la convenzione FLOPs/MAC~~ — **risolta** il 17/08: il paper riporta FLOPs, non MAC.
- ~~Verificare l'ipotesi del secondo strato convolutivo~~ — **confutata** il 18/08.
- ~~Verificare la formula di PhaseMix contro il codice di EAT~~ — fatto il 18/08: era sbagliata, ora è corretta.
- ~~Scrivere src/train.py~~ — fatto il 18/08, con resume trasparente e guardia sul test set.

Diario di bordo del progetto d'esame B031278 — Deep Learning, Fall 2025, Università di Firenze. Documento di lavoro personale, aggiornato a ogni sessione. Sorgente: `docs/diario_di_bordo.html` — rigenerare con `scripts/build_docs.ps1`.